MỞ ĐẦU

1. Lý do chọn đề tài

Trong kỷ nguyên kinh tế số và sự bùng nổ mạnh mẽ của thương mại điện tử (E-commerce) tại Việt Nam, dịch vụ giao hàng chặng cuối (Last-mile delivery) đã trở thành mắt xích sống còn, quyết định năng lực cạnh tranh và trải nghiệm khách hàng của các doanh nghiệp logistics. Mỗi ngày, các trung tâm điều hành vận tải phải xử lý hàng chục ngàn đến hàng triệu đơn hàng phân tán ngẫu nhiên trong không gian đô thị phức tạp. Thực tế hạ tầng giao thông tại các đô thị lớn như Hà Nội và TP. Hồ Chí Minh – với đặc trưng mật độ phương tiện cao, nhiều tuyến đường một chiều, hẻm nhỏ và các khung giờ cao điểm kẹt xe – đặt ra áp lực tối ưu hóa chi phí vận hành cực kỳ gay gắt.

Về mặt bản chất vận trù học, bài toán điều phối đội xe giao hàng đô thị được hình thức hóa thành **Bài toán Định tuyến Xe có Ràng buộc Sức chứa (Capacitated Vehicle Routing Problem - CVRP)**. Đây là bài toán tối ưu hóa tổ hợp kinh điển thuộc lớp **NP-Hard**. Khi quy mô đơn hàng n tăng lên, không gian trạng thái các phương án phân công tăng trưởng theo cấp giai thừa $O(n!)$. Việc sử dụng các phương pháp tính toán chính xác (Exact Methods như Branch-and-Bound, Branch-and-Cut) bị sụp đổ hoàn toàn về mặt thời gian (Computational Runtime Explosion) khi $n \geq 30$. Trong khi đó, các phương pháp kinh nghiệm cổ điển (Classical Heuristics như Savings, Sweep) tuy chạy nhanh nhưng lại vận hành theo tư duy tham lam (Greedy), dễ dàng bị mắc kẹt hoàn toàn tại các **"thung lũng tối ưu cục bộ" (Local Optima Trap)**, cho ra chất lượng lộ trình kém và lãng phí lớn tài nguyên phương tiện.

Để vượt qua rào cản tối ưu cục bộ, việc ứng dụng các thuật toán Siêu kinh nghiệm (Metaheuristics) – đặc biệt là **Thuật toán Di truyền (Genetic Algorithm - GA)** – là con đường tất yếu. Tuy nhiên, qua khảo sát thực tế kỹ thuật phần mềm tại Việt Nam, phần lớn các nghiên cứu GA hiện nay mới chỉ dừng lại ở các mô hình mô phỏng kịch bản tĩnh trên các công cụ nghiên cứu đơn luồng. Khi đưa vào môi trường sản xuất (Production), việc đánh giá độ thích nghi (Fitness Evaluation) qua hàng ngàn thế hệ tạo ra nút thắt cổ chai về độ trễ, phá hỏng cam kết phản hồi thời gian thực ($3 \div 5$ giây) của hệ thống Backend.

Xuất phát từ những thách thức lý thuyết và yêu cầu thực tiễn khắt khe đó, đề tài **"Nghiên cứu Thuật toán Di truyền giải Bài toán Định tuyến Xe trong Giao vận Đô thị và Triển khai Lõi Tối ưu hóa Backend (Optimization Backend Engine)"** được lựa chọn nghiên cứu. Đề tài không chỉ tập trung giải quyết mô hình toán học CVRP mà còn đóng vai trò là một công trình **Kỹ thuật Phần mềm (Software Engineering)**: Chuyển hóa giải thuật GA thành một **Microservice Tối ưu hóa Backend** xử lý đa luồng, tối ưu hóa bộ nhớ đệm CPU và sẵn sàng tích hợp vào hệ thống logistics thực tế.

2. Mục đích và nhiệm vụ nghiên cứu

2.1. Mục tiêu Lý thuyết

- Hệ thống hóa mô hình toán học Quy hoạch tuyến tính nguyên hỗn hợp (MILP) cho Bài toán Định tuyến Xe có Ràng buộc Sức chứa (CVRP) và các biến thể mở rộng trong giao vận đô thị (ràng buộc cửa sổ thời gian, tải trọng kĩ thuật).

- Phân tích sâu độ phức tạp tính toán NP-Hard, chứng minh hiện tượng "Bùng nổ tổ hợp" (Combinatorial Explosion) và cơ chế bứt phá bẫy Tối ưu cục bộ của Thuật toán Di truyền (GA).
- Nghiên cứu mô hình Di truyền Song song dạng Chủ - Tớ (Shared-Memory Master-Slave Parallel GA) và kỹ thuật tối ưu hóa bộ nhớ đệm CPU (CPU Cache Locality) trong kiến trúc vi xử lý đa nhân.

2.2. Mục tiêu Kỹ thuật Phần mềm và Triển khai

- Đề xuất và cài đặt **Cấu trúc dữ liệu Mã hóa Mảng 1 chiều phân tách số 0 (Delimiter-based Integer Encoding)**, giúp truy xuất bộ nhớ RAM với độ phức tạp $O(1)$ và thực thi **Thuật toán giải mã tuyến tính Single-Pass** $O(N)$.
- Xây dựng **Lõi phần mềm Tối ưu hóa Backend (Optimization Backend Engine)** xử lý đa luồng (Multithreading Thread Pool), tích hợp kỹ thuật Pool đối tượng (Object Pooling) để triệt tiêu chi phí thu gom rác (Garbage Collection Pauses).
- Đóng gói Lõi tối ưu thành một **Microservice độc lập** giao tiếp qua RESTful API chuẩn định dạng JSON, có khả năng tiếp nhận lô đơn hàng (Batching Window) và trả về phương án phân công lộ trình tối ưu trong **ngân sách thời gian** < 3 **giây**.
- Thực hiện đánh giá đối sánh định lượng trên bộ dữ liệu kiểm thử chuẩn quốc tế **CVRPLib**, đạt chỉ số độ lệch nghiệm $Gap < 2\%$ so với Best Known Solution (BKS).

3. Đối tượng và phạm vi nghiên cứu

3.1. Đối tượng Nghiên cứu

- Bài toán Định tuyến Xe có Ràng buộc Sức chứa (CVRP) trong logistics giao hàng chặng cuối đô thị.
- Thuật toán Di truyền (Genetic Algorithm - GA), các toán tử di truyền chuyên biệt (OX, Swap, Shift, Inversion/2-Opt) và mô hình Di truyền Song song (Parallel GA).
- Kiến trúc phần mềm Backend, kỹ thuật quản lý bộ nhớ đệm CPU (L1-L3 Cache), lập trình song song bộ nhớ chia sẻ và đóng gói RESTful API Microservice.

3.2. Phạm vi Nghiên cứu

- **Về không gian & hạ tầng:** Xét mô hình điều phối từ 01 Kho trung tâm (Single-Depot) đến n điểm giao hàng trong khu vực nội thành đô thị với đội xe đồng nhất (K phương tiện có sức chứa C_{cap}).
- **Về cơ chế xử lý:** Tập trung vào cơ chế **Điều phối theo lô thời gian cố định (Fixed-Time Batching Window)**, chuyển dòng đơn hàng động thành chuỗi các bài toán tối ưu tĩnh theo cửa sổ thời gian $30 \div 60$ giây.
- **Về dữ liệu thực nghiệm:** Thử nghiệm, đo lường và đánh giá đối sánh trên các tập dữ liệu chuẩn mực quốc tế thuộc thư viện **CVRPLib** (Augerat, Christofides, Uchoa Benchmark Instances) với quy mô từ $n = 32$ đến $n = 200 +$ khách hàng.

4. Phương pháp nghiên cứu

Đề tài kết hợp chặt chẽ giữa phương pháp nghiên cứu lý thuyết vận trù học và phương pháp thực nghiệm kỹ thuật phần mềm:

1. Phương pháp Nghiên cứu Lý thuyết & Phân tích Định lượng:
    - Sử dụng lý thuyết Đồ thị ($G = (V, E)$) và quy hoạch toán học để hình thức hóa hệ thống phương trình ràng buộc CVRP.
    - Trích xuất lý thuyết tiến hóa sinh học, phân tích cấu trúc không gian trạng thái và chứng minh độ phức tạp tính toán của thuật toán.
2. Phương pháp Thiết kế Kiến trúc Phần mềm & Tối ưu hóa Mức thấp:
    - Áp dụng nguyên lý thiết kế bộ nhớ đệm (Cache-Friendly Design) để xây dựng cấu trúc mảng primitive 1 chiều liên tục.
    - Sử dụng mô hình lập trình song song Chủ - Tớ (Master-Slave Parallelism) sử dụng bộ nhớ chia sẻ để phân phân phối tải tính toán Fitness trên các nhân CPU.
3. Phương pháp Thực nghiệm & Đánh giá Đối sánh (Benchmarking):
    - Lập trình hiện thực hóa Lõi Engine Backend trên ngôn ngữ lập trình hiệu năng cao (C++/Java).
    - Đo lường chỉ số %Gap so với nghiệm tối ưu Best Known Solution (BKS) trên CVRPLib, tính toán Hệ số tăng tốc song song (S_p), độ trễ phản hồi (T_{CPU}) và đánh giá tính ổn định qua các lần chạy ngẫu nhiên (Random Seed Stability).

5. Những đóng góp mới của đề tài

5.1. Ý nghĩa Khoa học

- Đóng góp một giải pháp mã hóa mảng 1 chiều phân tách số 0 tối ưu cho thuật toán Di truyền giải CVRP, giúp đơn giản hóa cấu trúc dữ liệu và rút ngắn độ phức tạp giải mã về mức tuyến tính $O(N)$.
- Hệ thống hóa phương pháp song song hóa GA trên kiến trúc CPU đa nhân bộ nhớ chia sẻ, chứng minh hiệu quả giảm độ trễ tính toán trong các bài toán tối ưu tổ hợp thời gian thực.

5.2. Ý nghĩa Thực tiễn

- Cung cấp một **Lõi phần mềm Tối ưu hóa Backend (Optimization Backend Engine)** hoàn chỉnh, có khả năng đóng gói thành Microservice thương mại để tích hợp trực tiếp vào các hệ thống Quản lý Vận tải (TMS), ứng dụng giao hàng nhanh hoặc nền tảng gọi xe công nghệ tại Việt Nam.
- Tự chủ công nghệ lõi điều phối nội bộ (In-house Optimization Engine), giúp các doanh nghiệp logistics Việt Nam tiết kiệm đáng kể chi phí bản quyền phần mềm ngoại, giảm $15\% \div 25\%$ tổng quãng đường di chuyển của đội xe, cắt giảm chi phí nhiên liệu và giảm thiểu lượng khí thải carbon ra môi trường đô thị.

6. Cấu trúc của ĐACN

Ngoài Phần Mở đầu, Bảng từ viết tắt, Danh mục sơ đồ/bảng biểu và Tài liệu tham khảo, nội dung chính của Đồ án được cấu trúc thành **3 Chương**:

- **CHƯƠNG 1: TỔNG QUAN VỀ HỆ THỐNG ĐIỀU PHỐI LOGISTICS ĐÔ THỊ VÀ BÀI TOÁN CVRP** Giới thiệu tổng quan quy trình điều phối logistics chặng cuối, phân tích thách thức vận hành, hình thức hóa mô hình toán học MILP cho CVRP, chứng minh định lý NP-Hard và sự bùng nổ tổ hợp. Khảo sát 3 hướng tiếp cận (Exact, Classical Heuristics, Metaheuristics), phân tích bẫy Tối ưu cục bộ và tổng quan các công trình liên quan trong/ngoài nước. Khảo sát thư viện dữ liệu chuẩn CVRPLib, các ràng buộc cửa sổ thời gian, kiến trúc đa luồng và thiết lập bộ tiêu chí đánh giá (Z, %Gap, T_{CPU}, S_p).

- **CHƯƠNG 2: THUẬT TOÁN DI TRUYỀN VÀ GIẢI PHÁP ĐỀ XUẤT (THIẾT KẾ LÕI OPTIMIZATION BACKEND ENGINE)** Trình bày chi tiết cơ sở lý thuyết Thuật toán Di truyền (GA) và luận điểm chuyển dịch sang Metaheuristic. Đề xuất cấu trúc dữ liệu Mã hóa Mảng 1 chiều phân tách số 0, xây dựng thuật toán giải mã tuyến tính Single-Pass $O(N)$ và phân tích tối ưu hóa Cache Locality. Thiết kế bộ toán tử di truyền chuyên biệt (k-Tournament, Modified OX, Swap, Relocate, Inversion/2-Opt), cơ chế Sửa nghiệm Chủ động và Hàm phạt Dynamic Penalty. Cài đặt chi tiết Mô hình Di truyền Song song Chủ - Tớ (Master-Slave Parallel GA), kỹ thuật Object Pooling và đóng gói RESTful API Microservice.

- **CHƯƠNG 3: KẾT QUẢ THỰC NGHIỆM VÀ ĐÁNH GIÁ ĐỐI SÁNH** Mô tả môi trường thử nghiệm phần cứng/phần mềm và bảng cấu hình tham số GA. Trình bày chi tiết kết quả thực nghiệm trên các tập dữ liệu chuẩn CVRPLib (Augerat, Christofides, Uchoa Instances). Phân tích định lượng chất lượng lộ trình (Z), độ chính xác (%Gap to BKS), thời gian thực thi (T_{CPU}) và hệ số tăng tốc đa luồng (S_p). Đánh giá độ ổn định qua các lần chạy hạt giống ngẫu nhiên (Random Seeds) và đối sánh hiệu năng với công cụ Google OR-Tools.

- **KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN** Tổng kết các kết quả đạt được của đồ án về mặt lý thuyết lẫn sản phẩm phần mềm Backend, thẳng thắn chỉ ra các hạn chế còn tồn tại và đề xuất các hướng phát triển mở rộng trong tương lai (Rich VRP, tích hợp Học máy Machine Learning/Reinforcement Learning).

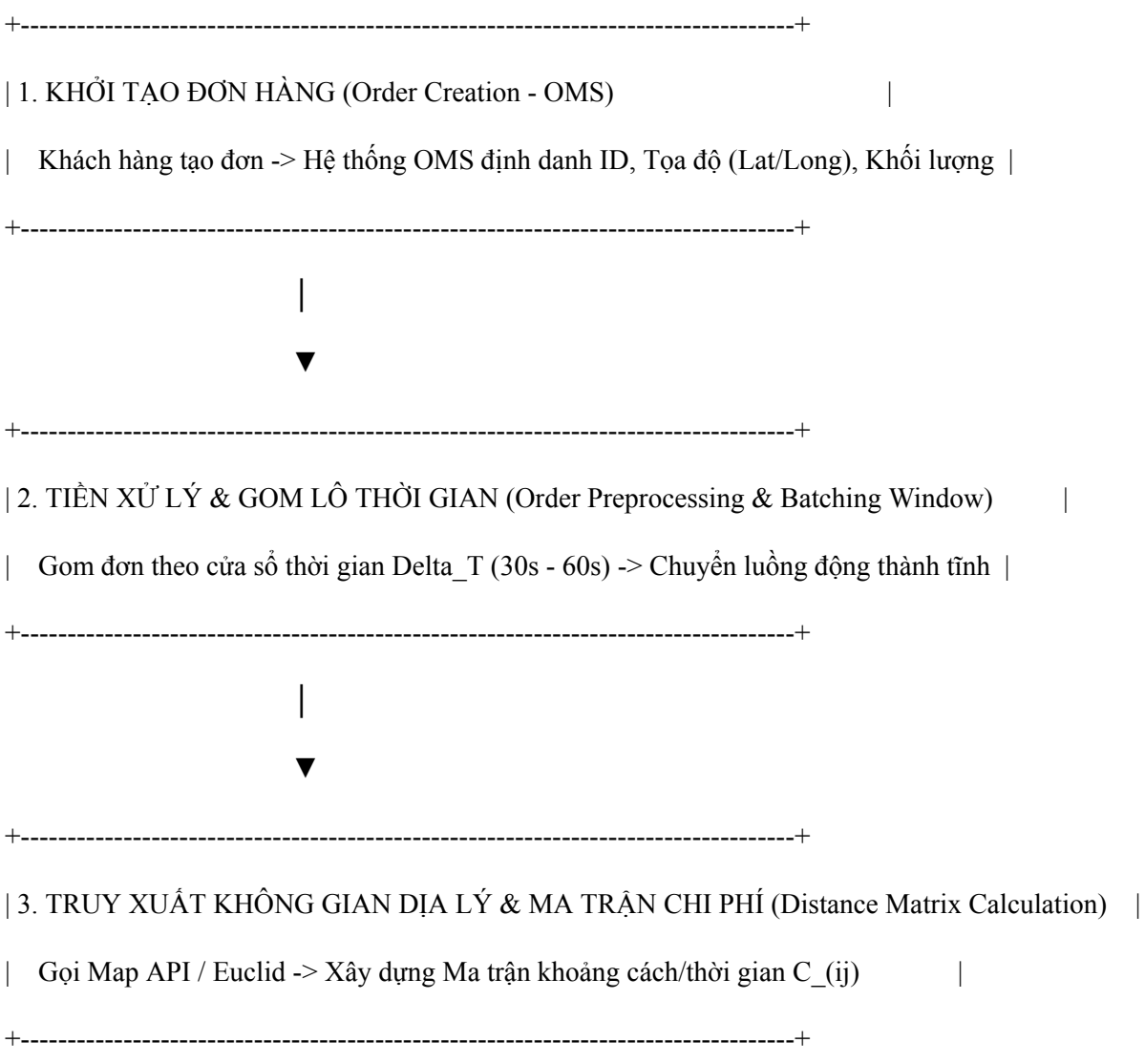
CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG ĐIỀU PHỐI LOGISTICS ĐÔ THỊ VÀ BÀI TOÁN CVRP

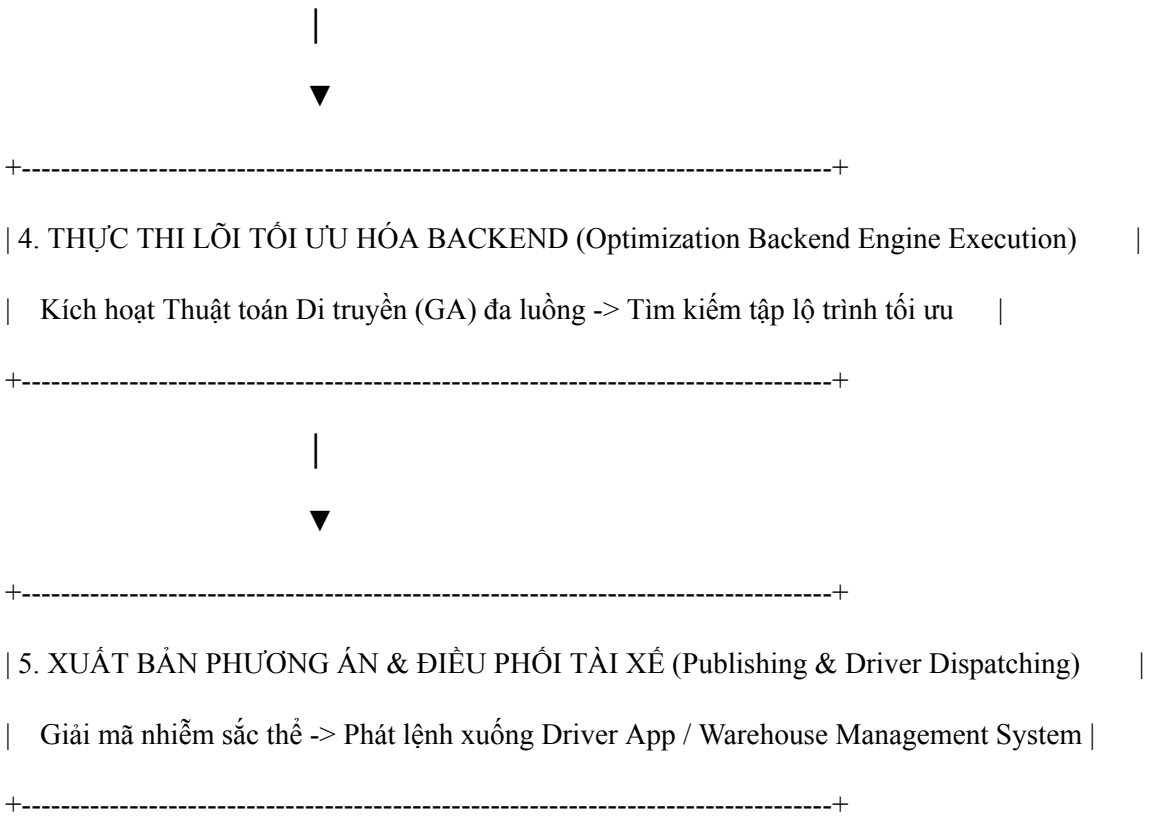
1.1. Tổng quan hệ thống điều phối logistics đô thị

1.1.1. Quy trình hoạt động của hệ thống (Flow từ lúc tạo đơn đến gán xe)

Trong bối cảnh bùng nổ thương mại điện tử (E-commerce) và dịch vụ giao hàng nhanh chặng cuối (Last-mile delivery) tại các đô thị hiện đại, hệ thống quản lý và điều phối vận tải (Transportation Management System - TMS) đóng vai trò là "bộ não" quyết định sự tồn tại và năng lực cạnh tranh của các doanh nghiệp logistics. Mục tiêu cốt lõi của quy trình điều phối là tiếp nhận liên tục chuỗi đơn hàng từ khách hàng, xử lý ràng buộc không gian - thời gian, và tự động hóa việc gán phương tiện vận tải sao cho tối thiểu hóa tổng chi phí vận hành trên toàn hệ thống.

Quy trình vận hành khép kín của một hệ thống điều phối logistics đô thị tự động được thiết kế bao gồm 5 giai đoạn nối tiếp chặt chẽ (Hình 1.1):





1.1.1.1. Giai đoạn 1: Khởi tạo đơn hàng (Order Creation)

Đơn hàng được khởi tạo từ phía người dùng cuối thông qua ứng dụng di động (Mobile App) hoặc giao diện Web tích hợp với Hệ thống Quản lý Đơn hàng (Order Management System - OMS). Mỗi đơn hàng i khi đi vào hệ thống được chuẩn hóa thành một bộ dữ liệu thuộc tính bao gồm: Mã định danh khách hàng (ID_i), tọa độ địa lý không gian (x_i, y_i), khối lượng/thể tích hàng hóa cần giao (q_i), và cửa sổ thời gian yêu cầu phục vụ $[e_i, l_i]$.

1.1.1.2. Giai đoạn 2: Tiền xử lý & Gom lô thời gian (Order Preprocessing & Batching Window)

Trong môi trường đô thị, các đơn hàng xuất hiện mang tính chất động và ngẫu nhiên theo thời gian. Để tránh việc gán xe tham lam (Greedy) từng đơn lẻ gây lãng phí hành trình, hệ thống áp dụng **Cơ chế gom lô thời gian cố định (Fixed-Time Batching)**. Dòng đơn hàng liên tục được tích lũy vào một đệm lưu trữ (Buffer) trong các khoảng thời gian ΔT ngắn (thường chọn $\Delta T = 30 \div 60$ giây). Tại thời điểm chốt Batch t, hệ thống đóng băng danh sách đơn hàng chưa giao và tập hợp đội xe đang rảnh rỗi tại kho gốc (Depot) để chuyển bài toán động thành chuỗi các bài toán tối ưu tĩnh tương đối.

1.1.1.3. Giai đoạn 3: Truy xuất không gian địa lý & Khởi tạo Ma trận Chi phí (Distance Matrix Generation)

Tọa độ không gian của kho trung tâm 0 và n điểm khách hàng trong lô được đưa vào bộ xử lý ma trận. Hệ thống tiến hành gọi các dịch vụ bản đồ (Google Maps API, OpenStreetMap) hoặc sử dụng

khoảng cách Euclid hình học trên mặt phẳng R^2 để khởi tạo Ma trận chi phí/khoảng cách hai chiều $C \in R^{(n+1)\times(n+1)}$, trong đó mỗi phần tử c_{ij} đại diện cho khoảng cách hoặc chi phí di chuyển trực tiếp từ điểm i đến điểm j.

1.1.1.4. Giai đoạn 4: Thực thi Lõi Tối ưu hóa Backend (Optimization Backend Engine Execution)

Ma trận chi phí c_{ij}, tập nhu cầu hàng hóa q_i, và tải trọng định mức của xe C_{cap} được đóng gói thành thông điệp chuẩn (JSON) và gửi tới **Microservice Tối ưu hóa Backend**. Tại đây, **Thuật toán Di truyền (Genetic Algorithm - GA)** được kích hoạt. Nhờ kiến trúc lập trình đa luồng (Multithreading), tiến trình chủ phân phân phối quần thể cho các luồng xử lý song song trên nhân CPU để thực hiện các phép toán tiến hóa (Chọn lọc, Lai ghép trật tự OX, Đột biến đảo chuỗi) và đánh giá hàm thích nghi qua hàng trăm thế hệ nhằm tìm ra phương án phân công lộ trình tiệm cận tối ưu toàn cục.

1.1.1.5. Giai đoạn 5: Xuất bản phương án & Điều phối tài xế (Publishing & Driver Dispatching)

Kết quả tối ưu dưới dạng chuỗi nhiễm sắc thể mảng 1 chiều được Engine Backend giải mã thành danh sách các lộ trình tuyến tính. Dịch vụ Backend xuất bản phương án gán xe, truyền dữ liệu lộ trình đến Hệ thống Quản lý Kho (WMS) để chuẩn bị xuất hàng lên xe, đồng thời tự động phát thông báo phân công cuốc xe (Dispatch Notification) xuống ứng dụng di động của từng tài xế (Driver App).

1.1.2. Thách thức trong điều phối (Tải trọng, Kẹt xe)

Bài toán điều phối giao vận tại các trung tâm đô thị lớn (như Hà Nội hay TP. Hồ Chí Minh) phải đối mặt với nhiều rào cản kỹ thuật khắt khe xuất phát từ đặc thù hạ tầng giao thông và mô hình vận tải thực tế:

1.1.2.1. Ràng buộc cứng về Tải trọng Kỹ thuật (Vehicle Capacity Constraint - C_{cap})

Mỗi phương tiện vận chuyển trong đội xe (dù là xe tải nhẹ hay xe máy giao hàng) đều có một sức chở hoặc tải trọng thiết kế tối đa cố định C_{cap}. Trong suốt hành trình xuất phát từ kho trung tâm, đi qua các trạm giao hàng và quay về kho, tổng khối lượng hàng hóa được xếp lên phương tiện k tuyệt đối không được phép vượt quá giới hạn C_{cap}: \[\sum_{i \in \text{Route}k} q_i \le C{cap}, \quad \forall k \in {1, \dots, K}\] Nếu thuật toán điều phối vi phạm ràng buộc này, phương tiện sẽ rơi vào trạng thái quá tải kĩ thuật, gây nguy cơ mất an toàn giao thông, vi phạm quy định pháp luật và có thể phải quay về kho xếp lại hàng, làm vỡ tiến độ của toàn bộ chuỗi cung ứng.

1.1.2.2. Sự biến động của Hạ tầng Giao thông Đô thị (Traffic Congestion & Dynamic Travel Times)

Khác với mô hình lý thuyết giả định khoảng cách địa lý là cố định và đối xứng ($c_{ij} = c_{ji}$), môi trường giao thông đô thị thực tế mang tính bất đối xứng, phi tuyến và biến động phức tạp theo thời gian. Mật độ giao thông thay đổi đột biến giữa các khung giờ cao điểm (Peak hours) và thấp điểm (Off-peak hours), dẫn đến vận tốc di chuyển $v(t)$ trên các tuyến đường biến thiên liên tục. Một chặng di chuyển từ i đến j vào lúc 10:00 sáng có thể chỉ mất 10 phút, nhưng vào lúc 17:30 chiều có thể kéo dài trên 35

phút do kẹt xe nghiêm trọng. Điều này đòi hỏi ma trận chi phí chi phối bài toán phải được cập nhật linh hoạt và thuật toán phải có khả năng dự phòng độ trễ hành trình.

1.1.2.3. Ràng buộc Cửa sổ Thời gian Phục vụ (Time Windows Constraints)

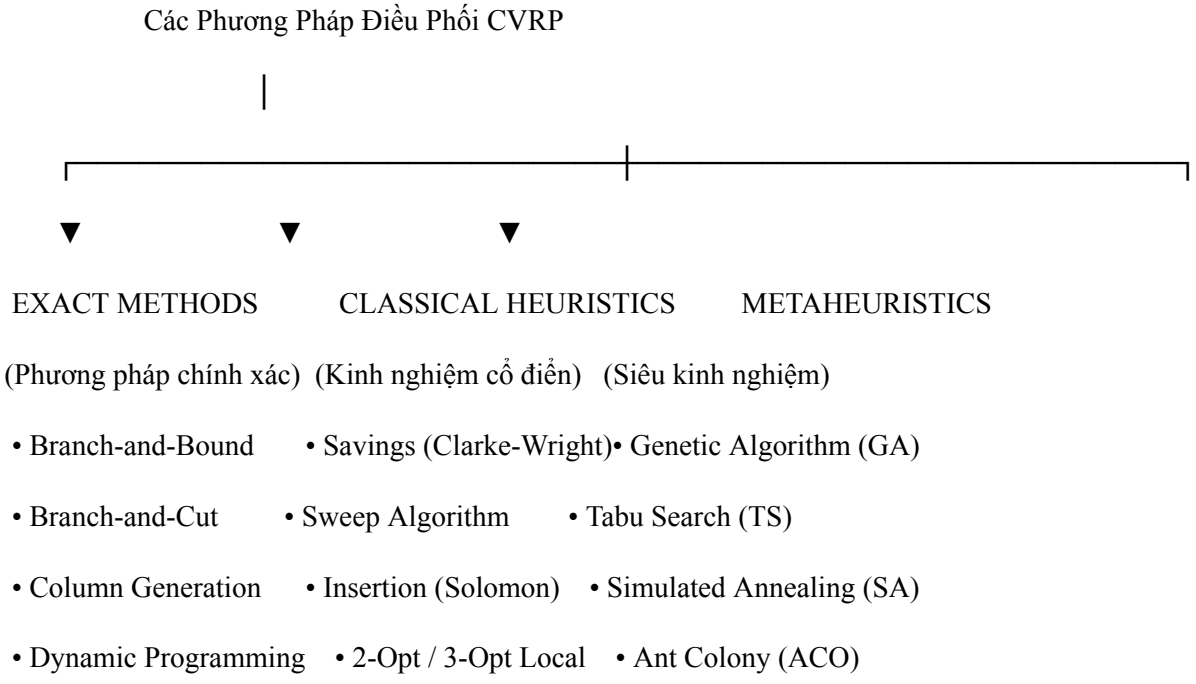
Áp lực phục vụ khách hàng đô thị đòi hỏi việc giao hàng phải diễn ra trong các khung giờ mong muốn xác định $[e_i, l_i]$ (trong đó e_i là thời điểm bắt đầu sớm nhất và l_i là thời điểm muộn nhất chấp nhận được). Việc phương tiện đến trước e_i buộc tài xế phải dừng chờ (phát sinh chi phí thời gian chờ w_i), trong khi việc đến sau l_i sẽ vi phạm cam kết chất lượng dịch vụ (SLA), phát sinh hệ số phạt tài chính hoặc khiến khách hàng từ chối nhận hàng.

1.1.2.4. Thách thức Tính toán Thời gian thực (Real-Time Computational Budget)

Trong kiến trúc phần mềm Backend, Engine tối ưu hóa bắt buộc phải đưa ra phương án phân công lộ trình hoàn chỉnh trong một cửa sổ thời gian cực kỳ ngắn (thường chỉ từ 3 đến 5 giây cho mỗi lô đơn hàng). Nếu thời gian tính toán của thuật toán kéo dài quá lâu, dữ liệu trạng thái giao thông và vị trí sẵn sàng của đội xe sẽ bị "lạc hậu" (stale data), dẫn đến phương án tính toán ra không thể áp dụng được vào thực tế.

1.2. Các phương pháp điều phối CVRP

Nhằm giải quyết Bài toán Định tuyến Xe có Ràng buộc Sức chứa (CVRP), giới khoa học máy tính và vận trù học đã phát triển hệ thống các phương pháp tiếp cận qua 3 giai đoạn tiến hóa:



1.2.1. Exact Methods (Phương pháp tối ưu chính xác)

Các phương pháp chính xác hướng tới mục tiêu duy nhất: Tìm ra nghiệm tối ưu toàn cục (Global Optimum) tuyệt đối của bài toán thông qua việc duyệt có hệ thống hoặc thu hẹp không gian trạng thái.

- **Các kỹ thuật tiêu biểu:** Nhánh và Cận (Branch-and-Bound), Nhánh và Cắt (Branch-and-Cut), Phân rã Lagrange (Lagrangian Decomposition), Sinh cột (Column Generation) và Quy hoạch động (Dynamic Programming).
- **Nguyên lý hoạt động:** Sử dụng các bài toán nới lỏng quy hoạch tuyến tính (LP Relaxation) để tính toán đường giới hạn dưới (Lower Bound), từ đó cắt tỉa (pruning) các nhánh lời giải không tiềm năng trong cây tìm kiếm.
- **Nhược điểm chí mạng:** Độ phức tạp tính toán tăng trưởng theo **hàm mũ** $O(2^n \cdot n^2)$. Các thực nghiệm học thuật khẳng định các thuật toán chính xác chỉ giải quyết được các bài toán cỡ nhỏ ($n \leq 15 \div 30$ điểm giao hàng). Khi $n \geq 100$, thời gian tính toán bùng nổ kéo dài hàng ngàn năm, hoàn toàn bất khả thi đối với các ứng dụng phần mềm điều phối đô thị.

1.2.2. Classical Heuristics (Phương pháp kinh nghiệm cổ điển)

Để đáp ứng yêu cầu thời gian thực, các phương pháp Heuristic cổ điển ra đời nhằm tìm kiếm lời giải khả thi chấp nhận được trong thời gian ngắn. Chúng được chia thành hai nhóm:

1. **Heuristic xây dựng lộ trình (Route Construction Heuristics):**
    - *Thuật toán Tiết kiệm (Savings Algorithm - Clarke & Wright, 1964):* Bắt đầu bằng việc gán mỗi khách hàng một tuyến riêng (Kho → i → Kho), sau đó liên tục gộp hai tuyến có chỉ số tiết kiệm khoảng cách $s_{ij} = c_{i0} + c_{0j} - c_{ij}$ lớn nhất nếu không vi phạm tải trọng.
    - *Thuật toán Quét góc (Sweep Algorithm - Gillet & Miller, 1974):* Phân cụm khách hàng bằng cách quay một tia xung quanh kho trung tâm theo góc cực, sau đó giải bài toán TSP cho từng cụm.
    - *Thuật toán Chèn (Insertion Heuristics - Solomon, 1987):* Khởi tạo lộ trình gốc và sử dụng thủ tục đẩy thời gian (Push-Forward Insertion Heuristic - PFIH) để chèn lần lượt từng khách hàng chưa gán vào vị trí có chi phí chèn thấp nhất.
2. **Heuristic cải thiện lộ trình dựa trên Tìm kiếm cục bộ (Local Search Improvement Heuristics):**
    - Áp dụng các toán tử biến đổi lân cận $N(x)$ như 2-Opt, 3-Opt (thay thế r cạnh), Relocate (dịch chuyển nút) hoặc Swap (đổi chỗ 2 nút) để cải thiện lời giải ban đầu.

1.2.2.1. Phân tích sâu về nhược điểm "Tối ưu cục bộ" (Local Optima Trap)

Nguyên lý vận hành cốt lõi của Heuristic cổ điển là **sự tham lam (Greedy Approach)**. Thuật toán chỉ chấp nhận các bước di chuyển giúp làm giảm chi phí hàm mục tiêu $f(x') < f(x)$.

Trong không gian tìm kiếm tổ hợp không lồi (non-convex) và gập ghềnh của bài toán CVRP, bề mặt hàm mục tiêu chứa vô số các "thung lũng". Khi thuật toán di chuyển xuống một thung lũng mà tất cả các nghiệm lân cận $x' \in N(x)$ đều có chi phí cao hơn điểm hiện tại x, thuật toán sẽ bị mắc kẹt hoàn toàn tại **Điểm tối ưu cục bộ (Local Optimum)**. Do không có cơ chế chấp nhận tạm thời các nghiệm

xấu hơn để "leo dốc" vượt qua rào cản không gian, Heuristic cổ điển cho ra chất lượng lộ trình kém và kém linh hoạt trước các bài toán biến động.

1.2.3. Metaheuristics (Phương pháp siêu kinh nghiệm)

Nhằm vượt qua bẫy tối ưu cục bộ của Heuristic cổ điển, các siêu thuật toán (Metaheuristics) đã được nghiên cứu và phát triển mạnh mẽ.

1.2.3.1. Nguyên lý thoát khỏi bẫy Tối ưu cục bộ của Metaheuristics

Metaheuristics đạt được thành công nhờ sự cân bằng thông minh giữa hai chiến lược: **Khai thác (Intensification/Exploitation - đào sâu lân cận tốt)** và **Khám phá (Diversification/Exploration - mở rộng tìm kiếm sang vùng không gian mới)**. Các thuật toán sở hữu cơ chế vượt bẫy chuyên biệt:

- **Thuật toán Luyện kim (Simulated Annealing - SA):** Chấp nhận nghiệm xấu hơn với một xác suất $P = e^{-\Delta E/T}$, cho phép thuật toán "nhảy" khỏi thung lũng cục bộ khi "nhiệt độ" T còn cao.
- **Tìm kiếm cấm (Tabu Search - TS):** Sử dụng **Danh sách Tabu (Tabu List)** để cấm thuật toán quay lại các trạng thái đã đi qua trong k bước gần nhất, buộc tiến trình tìm kiếm phải thâm nhập vào các vùng không gian hoàn toàn mới.
- **Thuật toán Di truyền (Genetic Algorithm - GA):** Quản lý một **Quần thể (Population)** gồm nhiều cá thể (lời giải) song song. Toán tử **Lai ghép (Crossover)** tích lũy các chuỗi gen (đoạn hành trình) ưu tú từ bố mẹ. Toán tử **Đột biến (Mutation)** thực hiện các xáo trộn ngẫu nhiên có định hướng (đổi vị trí, đảo chuỗi mảng), liên tục tiêm sự đa dạng di truyền vào quần thể để phá vỡ trạng thái hội tụ sớm, giúp hệ thống thoát bẫy tối ưu cục bộ một cách tự nhiên.

1.2.3.2. Bảng so sánh chi tiết giữa 3 nhóm phương pháp điều phối CVRP

Tiêu chí so sánh	Exact Methods (Phương pháp Chính xác)	Classical Heuristics (Kinh nghiệm Cổ điển)	Metaheuristics (Siêu kinh nghiệm - GA)
Tính tối ưu toán học	Đảm bảo nghiệm Tối ưu toàn cục (Global Optimum) tuyệt đối.	Không đảm bảo; chất lượng nghiệm phụ thuộc dữ liệu đầu vào.	Tiệm cận tối ưu toàn cục (Near-optimal, sai số $< 1 \div 2\%$).
Thời gian thực thi (Runtime)	Bùng nổ theo hàm mũ $O(2^n \cdot n^2)$; cực kỳ chậm.	Rất nhanh (mili-giây); phản hồi tức thì.	Nhanh ($1 \div 5$ giây); đáp ứng hoàn hảo chu kỳ Batching.

Quy mô xử lý tối đa (n)	Nhỏ ($n \leq 15 \div 30$ điểm giao hàng).	Trung bình và lớn ($n \leq 100 \div 200$ điểm).	Cực lớn ($n \geq 1000 +$ điểm khách hàng).
Nguy cơ mắc kẹt Tối ưu cục bộ	Không bị mắc kẹt (do duyệt kiệt cùng không gian).	**Rất cao (Rào cản chí mạng)** do tư duy tham lam.	**Cực thấp** nhờ cơ chế đột biến và tiến hóa quần thể.
Khả năng song song hóa Backend	Khó song song hóa hiệu quả trên kiến trúc đa nhân.	Hầu như chạy tuần tự; khó chia nhỏ tác vụ.	**Rất cao** (Rất dễ phân phân phối đánh giá Fitness đa luồng).
Độ phức tạp cài đặt phần mềm	Cực kỳ phức tạp (đòi hỏi thư viện giải MILP nặng).	Trung bình; dễ viết mã nguồn đơn giản.	Cấu trúc mô-đun rõ ràng; dễ đóng gói Microservice.

1.3. Khảo sát công trình liên quan

1.3.1. Hướng tiếp cận Quốc tế

Nghiên cứu về bài toán VRP và các biến thể có lịch sử phát triển lâu đời trên thế giới với nhiều cột mốc tri thức quan trọng:

1.3.1.1. Giai đoạn đặt nền móng và Heuristic cổ điển (1959 - 1980)

Dantzig và Ramser (1959) chính thức khởi xướng bài toán "Truck Dispatching Problem" – tiền thân của VRP. Tiếp đó, Clarke và Wright (1964) công bố Thuật toán Tiết kiệm (Savings Algorithm), tạo ra công cụ tính toán lộ trình tự động đầu tiên cho ngành vận tải. Solomon (1987) đã mở rộng bài toán sang miền ràng buộc cửa sổ thời gian (VRPTW) và xây dựng bộ dữ liệu chuẩn mực Solomon Benchmark nổi tiếng.

1.3.1.2. Giai đoạn bùng nổ Metaheuristics và Thuật toán Di truyền (1990 - 2015)

Holland (1975) và Goldberg (1989) hoàn thiện lý thuyết nền tảng về Thuật toán Di truyền (GA). Baker và Ayechew (2003) đã ứng dụng thành công GA thuần túy vào bài toán CVRP và chứng minh GA cho kết quả tương đương hoặc vượt trội hơn các giải thuật Tabu Search trên các bộ dữ liệu chuẩn. Prins (2004) đóng góp bước tiến lớn khi đề xuất thuật toán Memetic (GA kết hợp Local Search) dựa trên kỹ thuật mã hóa hoán vị không vách ngăn và giải mã bằng thuật toán Split. Gintaras Vaira (2014) trong luận án tiến sĩ tại Đại học Vilnius đã nghiên cứu chuyên sâu việc ứng dụng GA giải bài toán Rich VRP (RVRP) chứa nhiều ràng buộc hỗn hợp, đồng thời kết hợp thuật toán Dijkstra hai chiều song song để tăng tốc tính toán ma trận đường đi. Uchoa và các cộng sự (2017) đã chuẩn hóa và phát triển Thư viện **CVRPLib**, cung cấp các bộ dữ liệu kiểm thử quy mô lớn hiện đại.

1.3.1.3. Giai đoạn ứng dụng công nghiệp và Tối ưu hóa đa mục tiêu (2016 - Nay)

Deb và các cộng sự đã công bố thuật toán NSGA-II, mở ra kỷ nguyên giải quyết các bài toán VRP đa mục tiêu (cân bằng giữa chi phí, thời gian và sự công bằng). Trong lĩnh vực công nghiệp logistics đô thị, Qin và các cộng sự (2020) tại tập đoàn DiDi Chuxing đã công bố hệ thống điều phối cuốc xe quy mô lớn dựa trên cơ chế Batching kết hợp Học tăng cường (Reinforcement Learning) và Tối ưu tổ hợp, chứng minh tính vượt trội của việc gom lô điều phối so với cơ chế gán xe tức thời.

1.3.2. Hướng tiếp cận tại Việt Nam

Tại Việt Nam, các nghiên cứu về VRP/CVRP và Metaheuristic đã có sự chuyển dịch rõ rệt từ lý thuyết thuần túy sang hướng tiếp cận Kỹ thuật Phần mềm và ứng dụng thực tiễn:

1.3.2.1. Các nghiên cứu học thuật học vị và Tạp chí chuyên ngành

- **Nguyễn Việt Hân (2009 - Đại học Bách Khoa Hà Nội):** Trong luận văn Thạc sĩ *"Thuật toán di truyền song song giải bài toán VRP với hạn chế thời gian"*, tác giả đã nghiên cứu mô hình thuật toán di truyền song song (Parallel GA) dựa trên kiến trúc Chủ - Tớ (Master-Slave). Tác giả sử dụng thư viện truyền thông MPI để phân phân phối các thao tác lai ghép, đột biến và tính hàm thích nghi cho các tiến trình tớ, giúp rút ngắn đáng kể thời gian chạy chương trình trên hệ thống máy tính kết nối mạng (Cluster).
- **Cao Ngọc Ánh và Trần Bích Thảo (2023 - Trường ĐH Kinh tế - Kỹ thuật Công nghiệp):** Trong công trình *"Ứng dụng giải thuật Tabu Search trong giải bài toán định tuyến xe"*, các tác giả đã hiện thực hóa giải thuật Tabu Search trên ngôn ngữ R (RStudio) để giải quyết bài toán CVRP, thử nghiệm các cơ chế bộ nhớ cấm và hàm đánh giá chuyển dịch lân cận.
- **Thái Quốc Bảo (2026 - Trường Đại học Sài Gòn):** Trong báo cáo *"Nghiên cứu thuật toán Metaheuristic giải bài toán định tuyến xe trong giao vận đô thị"*, tác giả đã tập trung thiết kế cấu trúc dữ liệu mã hóa nhiễm sắc thể mảng 1 chiều phân tách số 0, ứng dụng thuật toán GA kết hợp cơ chế phạt vi phạm tải trọng linh hoạt và tích hợp công cụ đối sánh Google OR-Tools trên nền tảng Backend.
- **Lâm Thái Yến Nhi (2026 - Trường Đại học Sài Gòn):** Trong nghiên cứu *"Tiếp cận bằng thuật toán NSGA-II cho bài toán điều phối gọi xe đa mục tiêu theo cơ chế Batching có ràng buộc cửa sổ thời gian"*, tác giả đã xây dựng mô hình ghép cặp đa mục tiêu (tối thiểu quãng đường chạy rỗng và tối đa sự công bằng tài xế) trong môi trường điều phối theo lô động.

1.3.2.2. Tình hình ứng dụng thực tế trong công nghiệp Việt Nam

Các công ty công nghệ vận tải và logistics hàng đầu tại Việt Nam (như Giao Hàng Nhanh - GHN, Ahamove, Viettel Post, Grab Vietnam) đang chuyển đổi mạnh mẽ từ việc phân tuyến thủ công dựa trên kinh nghiệm điều phối viên sang việc tự chủ xây dựng các **Lõi tối ưu hóa nội bộ (In-house Optimization Engine)**. Tiêu biểu như nền tảng xMobility 5.0 do EMDDI phát triển (khởi đầu từ nhóm nghiên cứu tại Đại học Quốc gia Hà Nội) đã ứng dụng các thuật toán điều vận tự động cho hàng chục ngàn xe taxi và xe công nghệ trên toàn quốc.

1.3.3. So sánh và xác định vấn đề

1.3.3.1. Bảng tổng hợp đối sánh các công trình nghiên cứu liên quan

Tác giả / Công trình	Trọng tâm nghiên cứu	Phương pháp tiếp cận	Khoảng trống / Hạn chế kỹ thuật
Baker & Ayechew (2003)	Giải bài toán CVRP tĩnh bằng GA đơn lẻ.	Thuật toán Di truyền thuần túy (GA) mã hóa hoán vị.	Chưa đóng gói thành dịch vụ phần mềm; chưa hỗ trợ xử lý đa luồng Backend.
Prins (2004)	Tối ưu chất lượng lộ trình CVRP.	Memetic Algorithm (GA kết hợp Split Algorithm và Local Search).	Tốn chi phí tính toán cho thuật toán Split; chưa tối ưu bộ nhớ truy xuất tĩnh.
Nguyễn Việt Hân (2009)	Tăng tốc thời gian tính toán cho VRPTW.	Parallel GA dựa trên mô hình Chủ - Tớ sử dụng thư viện MPI.	Phụ thuộc vào hạ tầng phần cứng Cluster/MPI; chưa đóng gói RESTful API Microservice.
Vaira Gintaras (2014)	Giải quyết bài toán Rich VRP (RVRP).	GA kết hợp thuật toán Dijkstra hai chiều song song.	Cấu trúc dữ liệu nhiệm vụ phức tạp; độ trễ cao khi tích hợp vào hệ thống chặng cuối.
Cao Ngọc Ánh & c sự (2023)	Tìm kiếm lân cận vượt tối ưu cục bộ.	Tabu Search cài đặt trên môi trường tính toán RStudio.	RStudio chỉ phục vụ nghiên cứu kịch bản tĩnh; không thể triển khai sản thực tế.
Đề tài của đồ án	**Xây dựng Lõi tối ưu Backend (Optimization Engine) giải CVRP đô thị.**	**GA tối ưu mảng 1 chiều phân tách + Đa luồng CPU + CVRPLib Benchmarking.**	**Khắc phục toàn bộ các hạn chế trên:** Tối ưu bộ nhớ RAM $O(N)$, đóng gói Microservice, phản hồi thời gian thực.

1.3.3.2. Xác định rõ vấn đề nghiên cứu của Đề tài (Research Problem Formulation)

Từ việc khảo sát và so sánh các công trình trong và ngoài nước, đề tài xác định rõ khoảng trống kỹ thuật và vấn đề nghiên cứu cốt lõi:

> **Vấn đề nghiên cứu:** *Nghiên cứu chuyển hóa mô hình toán học CVRP và Thuật toán Di truyền (GA) thành một* ***Lõi phần mềm tối ưu hóa Backend (Optimization Backend***

Engine) *hoàn chỉnh. Lõi phần mềm này sử dụng cấu trúc dữ liệu* ***Mã hóa mảng 1 chiều phân tách số 0 (Delimiter-based Integer Encoding)*** *giúp tối ưu hóa bộ nhớ đệm CPU (Cache Locality), kết hợp* ***Kiến trúc xử lý song song đa luồng (Multithreading Parallel Evaluation)*** *để đánh giá quần thể. Hệ thống có khả năng tiếp nhận lô đơn hàng đô thị, tự động tính toán phương án phân công đạt chất lượng cao (Gap so với BKS của CVRPLib* $< 2\%$*) trong ngân sách thời gian phản hồi cực ngắn (*< 5 *giây), sẵn sàng tích hợp linh hoạt vào các hệ thống Logistics thực tế.*

1.4. Khảo sát hệ thống dữ liệu thực nghiệm chuẩn và tiêu chí đánh giá chất lượng thuật toán

1.4.1. Giới thiệu Thư viện Benchmark quốc tế CVRPLib và Cấu trúc tập dữ liệu

Để đánh giá tính đúng đắn, khả năng tối ưu và hiệu năng tính toán của một thuật toán Metaheuristic hay một Lõi phần mềm điều phối Backend (Optimization Engine), việc thử nghiệm trên các tập dữ liệu ngẫu nhiên hoặc tự sinh là không đủ căn cứ khoa học. Trong cộng đồng nghiên cứu Vận trù học (Operations Research) và Khoa học Máy tính quốc tế, hệ thống dữ liệu chuẩn **CVRPLib (Capacitated Vehicle Routing Problem Library)** do Eduardo Uchoa và các cộng sự tại Đại học PUC-Rio phát triển được thừa nhận là tiêu chuẩn đo lường chuẩn mực nhất.

CVRPLib đóng vai trò là kho lưu trữ tập trung các thể hiện bài toán (benchmark instances) được đóng góp bởi nhiều nhà nghiên cứu lừng danh qua các thời kỳ, tiêu biểu như Augerat (1995), Christofides và Eilon (1969), Fisher (1994), cùng bộ dữ liệu quy mô lớn hiện đại của Uchoa và cộng sự (2017). Điểm cốt lõi giúp CVRPLib trở thành "thước đo vàng" cho bài toán CVRP là với mỗi thể hiện bài toán (ví dụ: A-n32-k5, P-n50-k10, X-n101-k25), thư viện cung cấp công khai giá trị **Best Known Solution (BKS)** – nghiệm tối ưu tuyệt đối (hoặc nghiệm tốt nhất từng được tìm ra bởi các kỹ thuật hàng đầu thế giới). Điều này tạo ra một hệ quy chiếu chuẩn (Baseline) định lượng để đánh giá chính xác độ chênh lệch nghiệm của Thuật toán Di truyền (GA) được xây dựng trong đồ án.

Tệp dữ liệu chuẩn trong CVRPLib được quy chuẩn hóa dưới định dạng tệp văn bản mã nguồn mở .vrp chứa các trường thuộc tính cấu trúc địa lý và vận hành:

```
NAME : A-n32-k5
COMMENT : (Augerat et al., Min no of trucks: 5, Optimal value: 784)
TYPE : CVRP
DIMENSION : 32
EDGE_WEIGHT_TYPE : EUC_2D
CAPACITY : 100
NODE_COORD_SECTION
 1 38 46
 2 32 40
 ...
 32 52 64
DEMAND_SECTION
 1 0
 2 19
```

...
32 23
DEPOT_SECTION
1
-1
EOF

- **Thông tin cấu hình hệ thống (Metadata):** Bao gồm tên thể hiện (NAME), kiểu bài toán (TYPE: CVRP), tổng số đỉnh (DIMENSION: n+1), và sức chứa định mức của phương tiện (CAPACITY: C).
- **Tọa độ không gian (NODE_COORD_SECTION):** Cung cấp vị trí mặt phẳng Euclid $2D$ (x_i, y_i) của kho trung tâm (đỉnh 1 hoặc đỉnh 0) và n điểm giao hàng. Khoảng cách trực tiếp giữa hai nút i và j được tính bằng khoảng cách Euclid chuẩn:
- $d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$
-
- **Nhu cầu hàng hóa (DEMAND_SECTION):** Khối lượng hàng hóa $q_i \geq 0$ cần giao tại từng trạm khách hàng i ($q_{depot} = 0$).

1.4.2. Phân loại đặc tính địa lý và phân bố không gian của các Instances

Dựa trên hình thái phân bố không gian địa lý và cấu trúc ràng buộc thời gian/tải trọng, các tập dữ liệu thực nghiệm chuẩn được chia thành các nhóm đặc trưng:

1.4.2.1. Nhóm C (Clustered - Phân cụm mật độ cao)

Các điểm giao hàng tập trung thành từng cụm địa lý riêng biệt (ví dụ: các trung tâm thương mại, khu công nghiệp tập trung). Kho trung tâm có thể nằm ở giữa hoặc lệch hẳn về một phía.

- *Đặc tính vận hành:* Dễ dàng gom cụm phương tiện, các thuật toán Heuristic truyền thống dựa trên góc cực (Sweep) thường đạt hiệu quả tương đối tốt trên nhóm này.

1.4.2.2. Nhóm R (Random - Phân tán ngẫu nhiên)

Tọa độ các điểm giao hàng được phân bố hoàn toàn ngẫu nhiên và phân tán đều trên mặt phẳng không gian.

- *Đặc tính vận hành:* Không xuất hiện cụm rõ rệt. Đây là nhóm thách thức cao đối với các thuật toán tham lam vì không gian tìm kiếm lân cận rộng lớn và gập ghềnh, đòi hỏi thuật toán phải có khả năng tìm kiếm toàn cục mạnh mẽ.

1.4.2.3. Nhóm RC (Random-Clustered - Hỗn hợp)

Tập hợp dữ liệu vừa chứa các cụm mật độ cao ở khu vực trung tâm, vừa chứa các điểm giao hàng lẻ rải rác ở vùng ven đô thị.

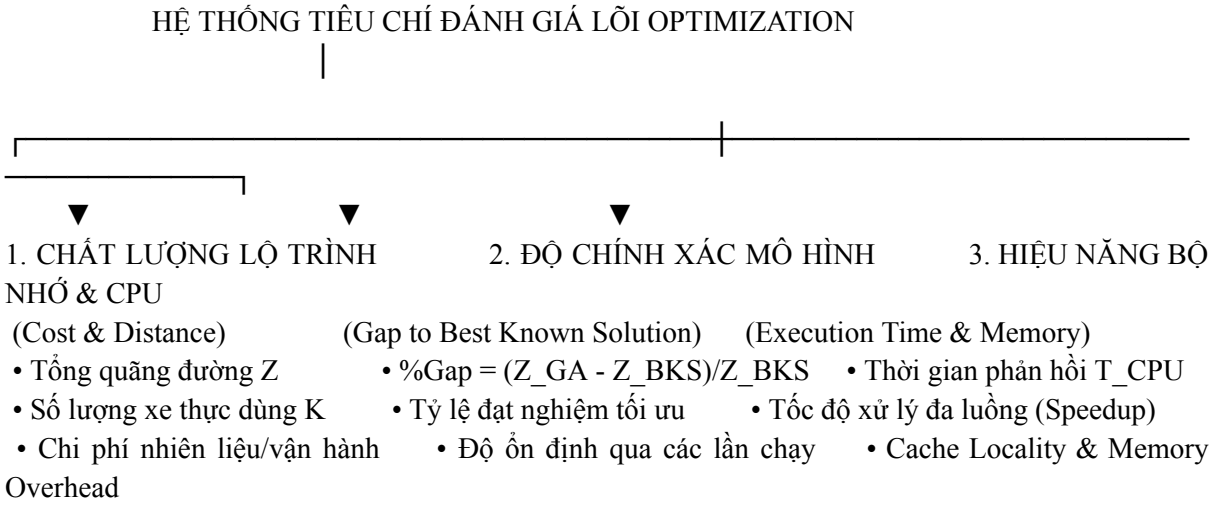
- *Đặc tính vận hành:* Phản ánh chân thực nhất hạ tầng giao thông đô thị thực tế (như Hà Nội hay TP. Hồ Chí Minh). Đòi hỏi Lõi tối ưu hóa Backend phải xử lý linh hoạt việc kết hợp giao hàng gom cụm và giao hàng đường dài.

1.4.2.4. Phân loại theo Quy mô Bài toán (Instance Scale)

- **Quy mô nhỏ ($n < 50$):** Dùng để kiểm thử tính đúng đắn logic của thuật toán (Unit Test) và đối sánh với nghiệm chính xác từ nhánh và cận (Branch-and-Bound).
- **Quy mô trung bình ($50 \leq n \leq 200$):** Quy mô tiêu chuẩn tương đương lô đơn hàng chặng cuối trong một cửa sổ Batching của quận/huyện đô thị.
- **Quy mô lớn ($n > 200 \div 1000 +$):** Kiểm thử năng lực chịu tải (Stress Test), khả năng quản lý bộ nhớ đệm CPU và tính hiệu quả của cơ chế đa luồng song song trên Engine Backend.

1.4.3. Tiêu chí đánh giá chất lượng thuật toán và hiệu năng phần mềm Backend

Khi đóng gói Thuật toán Di truyền (GA) thành một **Microservice Tối ưu hóa Backend (Optimization Backend Engine)**, việc đo lường chất lượng phần mềm đòi hỏi sự kết hợp chặt chẽ giữa chỉ số toán học tối ưu và các tiêu chí kỹ thuật phần mềm thời gian thực:



1.4.3.1. Tiêu chí 1: Tổng chi phí / Quãng đường di chuyển (Z)

Tiêu chí hàm mục tiêu cốt lõi của bài toán CVRP, biểu diễn tổng chiều dài quãng đường lăn bánh của toàn bộ đội xe:

$$Z = \sum_{k=1}^{K} \sum_{i \in V} \sum_{j \in V} c_{ij} x_{ijk}$$

Giá trị Z càng nhỏ trực tiếp thể hiện khả năng tiết kiệm nhiên liệu, giảm lượng khí thải carbon và tối ưu hóa tài sản phương tiện của doanh nghiệp.

1.4.3.2. Tiêu chí 2: Độ lệch so với Nghiệm tối ưu chuẩn (%Gap to BKS)

Chỉ số %Gap phản ánh năng lực bứt phá khỏi "tối ưu cục bộ" và độ chính xác của Thuật toán Di truyền so với đáp án tối ưu tuyệt đối trên CVRPLib:

$$Gap(\%) = \frac{Z_{GA} - Z_{BKS}}{Z_{BKS}} \times 100\%$$

Trong đó:

- Z_{GA}: Tổng quãng đường do Thuật toán Di truyền của đề tài tìm được.
- Z_{BKS}: Giá trị Best Known Solution công bố trên CVRPLib.

Đánh giá: Giá trị $Gap(\%)$ càng tiến gần về 0% chứng tỏ thuật toán đạt độ chính xác càng cao. Trong kỹ thuật phần mềm thương mại, chỉ số $Gap < 1.5\% \div 2\%$ trên các tập dữ liệu lớn đã đủ điều kiện để triển khai vào hệ thống sản xuất thực tế (Production).

1.4.3.3. Tiêu chí 3: Thời gian thực thi và Phản hồi hệ thống (T_{CPU})

Đo lường tổng thời gian tính toán (tính bằng giây hoặc mili-giây) từ lúc Microservice tiếp nhận thông điệp JSON chứa Ma trận chi phí đến khi trả về kết quả cấu trúc tuyến đường.

Trong kiến trúc Backend điều phối theo lô (Batching Window), chỉ số này phải thỏa mãn điều kiện ngân sách thời gian khắt khe:

$$T_{CPU} \leq T_{Batch_{B}udget} (thường chọn T_{CPU} \leq 3 \div 5 giây)$$

1.4.3.4. Tiêu chí 4: Hiệu năng xử lý song song (Parallel Speedup & Memory Overhead)

- **Hệ số tăng tốc (Speedup S_p):** Đo lường hiệu quả của kiến trúc lập trình đa luồng (Multithreading) trên CPU đa nhân:
- $S_p = \frac{T_1}{T_p}$
- *(Trong đó T_1 là thời gian chạy đơn luồng tuần tự, T_p là thời gian chạy song song trên p luồng CPU).*
- **Độ tối ưu bộ nhớ đệm (Cache Overhead):** Đảm bảo cấu trúc mảng 1 chiều phân tách số 0 duy trì chỉ số Cache Miss ở mức tối thiểu, không gây ra hiện tượng nghẽn bộ nhớ RAM hoặc ép bộ thu gom rác (Garbage Collector) hoạt động quá tải.

1.4.3.5. Bảng tổng hợp các chỉ số đánh giá chất lượng Lõi phần mềm Backend

Nhóm tiêu chí	Tên chỉ số	Ký hiệu	Công thức / Đơn vị	Ý nghĩa thực tiễn
Chất lượng Lộ trình	Tổng quãng đường	Z	$\sum c_{ij} x_{ijk}$ (km/đơn vị khoảng cách)	Đo lường trực tiếp chi phí vận tải và nhiên liệu.

Độ chính xác Toán học	Độ lệch BKS	%Gap	$\frac{Z_{GA}-Z_{BKS}}{Z_{BKS}} \times 100\%$	Đánh giá năng lực thoát bẫy tối ưu cục bộ.
Hiệu năng Hệ thống	Thời gian phản hồi	T_{CPU}	Giây (s) hoặc Mili-giây (ms)	Đảm bảo tính khả thi cho xử lý thời gian thực.
Hiệu năng Phần cứng	Hệ số tăng tốc	S_p	$S_p = T_1/T_p$	Đo lường hiệu quả tận dụng phần cứng CPU đa nhân.
Mức độ Ổn định	Độ lệch chuẩn nghiệm	σ_Z	$\sqrt{\frac{1}{M-1}\sum(Z_m - \bar{Z})^2}$	Đánh giá tính ổn định của thuật toán qua M lần chạy hạt giống.

1.9. Tóm tắt Chương 1

Chương 1 đã xây dựng một cách toàn diện và chi tiết bức tranh tổng quan về Hệ thống Điều phối Logistics Đô thị và Bài toán Định tuyến Xe có Ràng buộc Sức chứa (CVRP):

1. **Phát biểu & Hình thức hóa Bài toán:** Đã định nghĩa chặt chẽ bài toán CVRP trên đồ thị $G = (V, E)$, thiết lập hệ thống phương trình Quy hoạch tuyến tính nguyên hỗn hợp (MILP) với đầy đủ các ràng buộc phục vụ duy nhất, bảo toàn luồng, giới hạn tải trọng C_{cap} và loại bỏ chu trình con MTZ.
2. **Chứng minh định lý & Bùng nổ tổ hợp:** Đã chứng minh định lý CVRP thuộc lớp NP-Hard bằng phép quy về bài toán TSP. Ví dụ thực nghiệm toán học chỉ ra rằng với $n = 100$ điểm giao hàng, không gian trạng thái $99! \approx 9.33 \times 10^{155}$ vượt xa tổng số nguyên tử trong Vũ trụ quan sát được, khẳng định sự sụp đổ tuyệt đối của các phương pháp vét cạn và chính xác.
3. **Khảo sát & So sánh 3 hướng tiếp cận:** Đã phân tích chi tiết sự tiến hóa từ Exact Methods đến Classical Heuristics và Metaheuristics. Phân tích chỉ ra nhược điểm chí mạng "Tối ưu cục bộ" (Local Optima Trap) của các thuật toán kinh nghiệm tham lam cổ điển (Savings, Sweep) và chứng minh khả năng vượt bẫy thông minh của Thuật toán Di truyền (GA) nhờ cơ chế tiến hóa quần thể, lai ghép trật tự và đột biến ngẫu nhiên.
4. **Định hình vấn đề dưới góc nhìn Kỹ thuật Phần mềm Backend:** Khảo sát các công trình trong và ngoài nước đã làm rõ khoảng trống kỹ thuật: Vấn đề của đề tài không phải là phát minh công thức toán học mới, mà là **đóng gói Thuật toán Di truyền thành một Lõi tối ưu hóa Backend (Optimization Backend Engine / Microservice)** xử lý đa luồng, tối ưu hóa bộ nhớ đệm CPU bằng mã hóa mảng 1 chiều phân tách số 0, đáp ứng chu kỳ gom lô Batching thời gian thực.
5. **Nền tảng thực nghiệm chuẩn mực:** Đã thiết lập hệ thống kiểm thử dựa trên thư viện chuẩn quốc tế CVRPLib và bộ chỉ số định lượng bao gồm Z, %Gap to BKS, T_{CPU} và Speedup S_p.

Toàn bộ nền tảng lý luận, mô hình toán học và tiêu chí đo lường của Chương 1 tạo tiền đề vững chắc cho việc thiết kế kiến trúc phần mềm và cài đặt chi tiết Thuật toán Di truyền trong **Chương 2**.

1.5. Ràng buộc cửa sổ thời gian và các tham số thời gian trong điều phối

Trong hệ thống điều phối giao vận đô thị và logistics chặng cuối, thời gian là một trong những tham số vận hành cốt lõi quyết định trực tiếp đến chất lượng dịch vụ (SLA) và tính khả thi của lộ trình. Khác với các mô hình định tuyến tĩnh thuần túy chỉ tập trung tối thiểu hóa khoảng cách địa lý, việc tích hợp **Ràng buộc Cửa sổ Thời gian (Time Windows Constraints)** đòi hỏi hệ thống Backend phải kiểm soát chính xác độ trễ tích lũy từ khâu phát sinh đơn hàng, thời gian chờ xử lý lô (Batching), thời gian tài xế di chuyển đi đón, cho đến thời điểm giao hàng thực tế tại từng trạm khách hàng.

Để hình thức hóa các điều kiện vận hành này vào mô hình toán học và Lõi tối ưu hóa Backend, hệ thống thiết lập bộ tham số thời gian chuẩn hóa:

Bảng 1.2. Tổng hợp các tham số thời gian cốt lõi trong mô hình điều phối Backend

Tham số thời gian	Ký hiệu	Giá trị thiết lập dự kiến	Ý nghĩa vận hành trong hệ thống
Chu kỳ gom lô Batching	ΔT	$30 \div 60$ giây	Khoảng thời gian đệm để tích lũy dòng đơn hàng ngẫu nhiên trước khi chốt dữ liệu xử lý.
Ngân sách thời gian xử lý	b	$3 \div 5$ giây	Thời gian tính toán tối đa cho phép đối với Engine Backend để hoàn thành thuật toán.
Thời gian chờ phân công tối đa	W_{max}	300 giây (5 phút)	Giới hạn thời gian tối đa một đơn hàng được phép nằm trong hệ thống chờ gán xe.
Thời gian tiếp cận đón tối đa	H_{max}	1200 giây (20 phút)	Giới hạn từ khi phát sinh đơn hàng đến khi phương tiện dự kiến đến trạm giao nhận.
Thời gian phục vụ tại trạm	s_i	$180 \div 300$ giây	Thời gian tài xế dừng xe, xếp dỡ hàng hóa và thực hiện thủ tục xác nhận giao hàng.

1.5.1. Thời gian chờ phân công và Ngân sách xử lý Batch ($T_{assignment}$ & b)

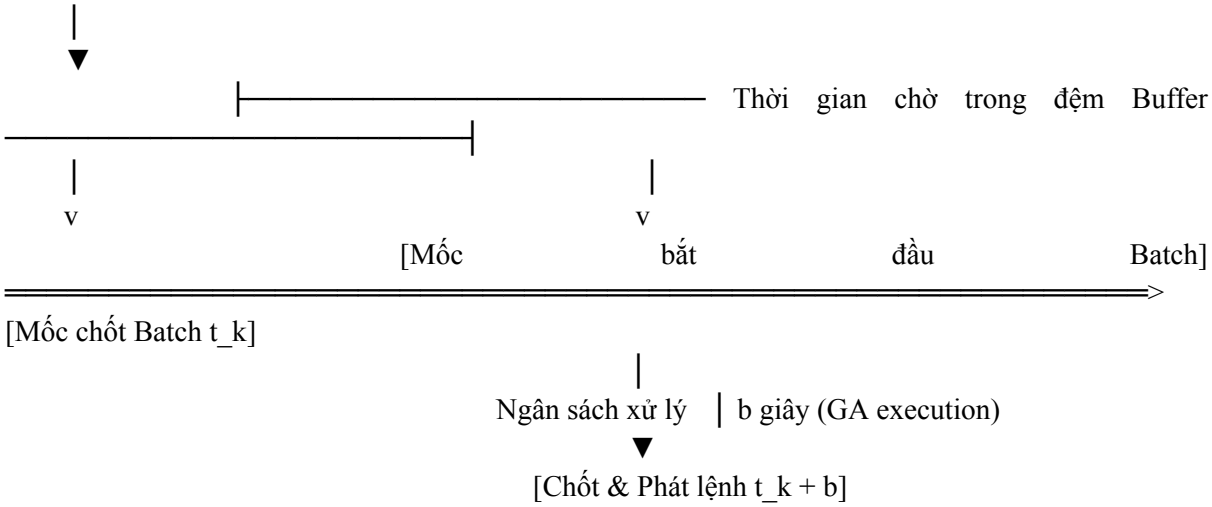
Thời gian chờ phân công (Assignment Waiting Time - $T_{assignment}$) là khoảng thời gian đo lường từ thời điểm khách hàng/hệ thống khởi tạo đơn hàng a_i đến khi Lõi tối ưu Backend chốt phương án gán xe và phát thông báo điều phối tới tài xế.

Trong cơ chế **Điều phối theo lô thời gian cố định (Fixed-Time Batching)**, dòng đơn hàng liên tục được gom vào các cửa sổ thời gian ΔT. Tại mốc chốt lô thứ k (ký hiệu t_k), hệ thống đóng băng danh sách đơn hàng đang chờ R_k và tập hợp đội xe sẵn sàng D_k để tiến hành giải bài toán ghép cặp. Để đảm bảo tính đồng bộ thời gian và công bằng cho thuật toán, hệ thống dành một **Ngân sách xử lý**

(Computational Budget) cố định b giây (với $0 < b < \Delta T$) để thực hiện việc truy xuất ma trận khoảng cách, kích hoạt Thuật toán Di truyền (GA) đa luồng và chọn lọc nghiệm. Thông báo phân công chính thức được xuất bản xuống ứng dụng của tài xế chính xác tại thời điểm $t_k + b$.

Ràng buộc thời gian chờ phân công của đơn hàng thứ i (phát sinh tại thời điểm a_i) bắt buộc phải thỏa mãn bất phương trình:

$$t_k + b - a_i \leq W_{max}, \forall i \in R_k (W_{max} = 300 giây)$$



Ý nghĩa kỹ thuật phần mềm Backend:

- Nếu một đơn hàng chưa được gán xe tại Batch k nhưng thời gian chờ $t_k + b - a_i$ vẫn nhỏ hơn W_{max}, đơn hàng đó tiếp tục được giữ lại trong đệm lưu trữ để chuyển sang xét duyệt tại Batch $k + 1$.
- Ngược lại, nếu tại mốc $t_k + b$, đơn hàng có thời gian chờ vượt quá W_{max} (300 giây) mà vẫn không tìm được phương án gán xe hợp lệ, đơn hàng đó sẽ bị đánh dấu **Hết hạn (Expired)** và chuyển sang quy trình xử lý ngoại lệ (ví dụ: tăng giá cước khuyến khích hoặc hủy chuyến) để tránh làm tắc nghẽn bộ nhớ đệm của Backend Engine.

1.5.2. Thời gian di chuyển đi đón và Thời gian phục vụ tại trạm (T_{pickup} & s_i)

Thời gian di chuyển đi đón/tiếp cận (Pickup Travel Time - τ_{ij}) đại diện cho khoảng thời gian phương tiện k di chuyển từ vị trí hiện tại (hoặc từ trạm giao hàng i trước đó) đến vị trí nhận hàng j.

Để đảm bảo cam kết chất lượng dịch vụ (SLA) đối với khách hàng đô thị, hệ thống quy định **Ràng buộc hạn đón tối đa (Maximum Arrival Deadline - h_j)** cho mỗi điểm giao hàng j:

$h_j = a_j + H_{max} (H_{max} = 1200 giây)$

Khi tài xế i được phân công di chuyển trực tiếp đến trạm j, thời điểm phương tiện tiếp cận trạm j phải thỏa mãn bất phương trình ràng buộc thời gian muộn nhất:

$$t_k + b + \tau_{ij} \leq h_j, \forall (i,j) \in E_k$$

Ngoài thời gian di chuyển trên đường τ_{ij}, tại mỗi trạm khách hàng j, hệ thống bổ sung tham số **Thời gian phục vụ cố định** s_j (Service Time) bao gồm thời gian xe dừng đậu, kiểm tra chứng từ và giao nhận hàng hóa. Sự dịch chuyển thời gian tích lũy (Push-Forward Time) khi phương tiện di chuyển nối tiếp qua chuỗi các điểm giao hàng trên cùng một tuyến được tính toán theo công thức truy hồi:

$$t_j^{arrival} = \max \left(e_j, t_i^{arrival} + s_i + \tau_{ij} \right)$$

Trong đó:

- $t_i^{arrival}$: Thời điểm xe bắt đầu tiếp cận trạm i.
- e_j: Thời điểm sớm nhất trạm j sẵn sàng mở cửa nhận hàng (Cửa sổ thời gian bên trái). Nếu xe đến sớm hơn e_j, tài xế bắt buộc phải dừng chờ một khoảng $w_j = e_j - (t_i^{arrival} + s_i + \tau_{ij})$ mà không phát sinh chi phí giao hàng.
- l_j: Thời điểm trễ nhất trạm j chấp nhận nhận hàng (Cửa sổ thời gian bên phải). Điều kiện tính khả thi bắt buộc $t_j^{arrival} \leq l_j$.

1.5.3. Ảnh hưởng của ràng buộc thời gian đến Không gian tìm kiếm và Kết quả phân công

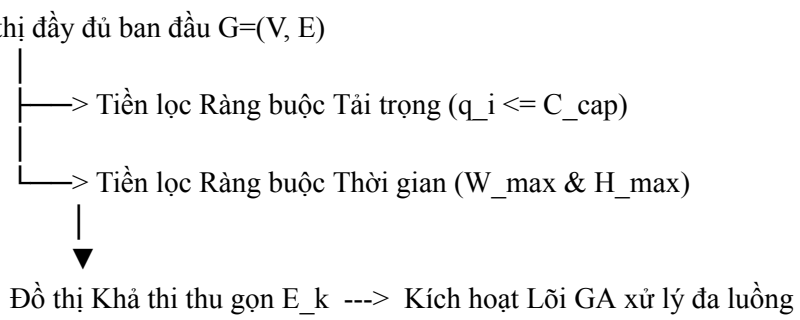
Việc đưa hệ thống ràng buộc thời gian hẹp vào bài toán CVRP tạo ra những tác động kỹ thuật sâu sắc tới cấu trúc không gian trạng thái và chiến lược thiết kế thuật toán trên Backend:

1.5.3.1. Tác động thu hẹp Đồ thị Khả thi (Feasibility Graph Reduction)

Trong bài toán CVRP lý thuyết, đồ thị giao thông $G = (V, E)$ được giả định là đồ thị đầy đủ. Tuy nhiên, dưới sự chi phối của hai điều kiện thời gian (W_{max} và H_{max}), Lõi Backend thực hiện một bước **Tiền lọc khả thi (Pre-filtering)**. Bất kỳ cặp kết nối (i, j) nào vi phạm một trong các điều kiện:

1. Đơn hàng j đã quá hạn chờ phân công ($t_k + b - a_j > W_{max}$).
2. Thời gian tiếp cận dự báo vượt quá thời hạn cho phép ($t_k + b + \tau_{ij} > h_j$).

Sẽ ngay lập tức bị loại bỏ khỏi tập cạnh hợp lệ E_k. Thao tác này giúp thu hẹp đáng kể kích thước ma trận chi phí, giảm bậc của các đỉnh và cắt tỉa phần lớn các nhánh lời giải không khả thi trước khi kích hoạt Thuật toán Di truyền.



1.5.3.2. Sự xung đột giữa Mục tiêu Tối ưu Chi phí và Ràng buộc Thời gian (Trade-off Analysis)

Ràng buộc cửa sổ thời gian đóng vai trò là "màng lọc khắt khe" đối với hàm mục tiêu. Trong nhiều trường hợp, một phương án phân công lộ trình có tổng quãng đường di chuyển Z cực kỳ ngắn (rất tối ưu về chi phí nhiên liệu) nhưng lại bị hệ thống từ chối loại bỏ vì có một điểm giao hàng trên tuyến bị trễ hạn cửa sổ thời gian l_j chỉ trong vài giây. Điều này chứng minh rằng việc chỉ đuổi theo mục tiêu ngắn nhất mà bỏ qua ràng buộc thời gian sẽ làm hỏng toàn bộ phương án vận hành.

1.5.3.3. Chiến lược xử lý Ràng buộc trong Thuật toán Di truyền (GA Repair & Penalty)

Để tránh việc các toán tử lai ghép (OX) và đột biến (Swap/Inversion) liên tục sinh ra các cá thể "phế phẩm" vi phạm cửa sổ thời gian (gây lãng phí tài nguyên tính toán CPU), Lõi phần mềm Backend đề xuất áp dụng **Cơ chế Sửa nghiệm Chủ động (Active Repair Mechanism)** dựa trên thuật toán chèn PFIH hoặc **Hàm phạt biến đổi linh hoạt (Dynamic Penalty Function)** vào hàm thích nghi:

\[Fitness(X) = Z + \alpha \cdot \sum_{k=1}^{K} \max\left(0, \sum_{i \in \text{Route}}*k* q_i - C{cap}\right) + \beta \cdot \sum_{j \in C} \max\left(0, t_{j}^{arrival} - l_j\right)\]

Trong đó β *là hệ số phạt vi phạm thời gian.*

Cơ chế này cho phép thuật toán duy trì tính liên tục của không gian tìm kiếm, hỗ trợ các chuỗi gen ưu tú "băng qua" vùng vi phạm thời gian ngắn hạn để hội tụ về vùng tối ưu toàn cục khả thi trong thời gian phản hồi thời gian thực $b \leq 3 \div 5$ giây của máy chủ Backend.

1.6. Kiến trúc xử lý đa luồng và tối ưu hóa bộ nhớ backend trong các bài toán tối ưu tổ hợp

1.6.1. Phân mảnh bộ nhớ (Memory Fragmentation) và Hiệu ứng Cache Locality trên CPU đa nhân

Trong kỹ thuật phần mềm Backend dành cho các ứng dụng tính toán hiệu năng cao (High-Performance Computing - HPC), việc thực thi các thuật toán Siêu kinh nghiệm (Metaheuristics) như Thuật toán Di truyền (GA) trên không gian bài toán quy mô lớn ($n \geq 100 \div 1000$ điểm giao hàng) đặt ra thách thức lớn về mặt quản lý bộ nhớ đệm CPU (CPU Cache).

Đối với các ngôn ngữ lập trình hiện đại (Java, C++, C#), nếu lập trình viên biểu diễn nhiễm sắc thể bằng các cấu trúc dữ liệu hướng đối tượng dựa trên con trỏ như Danh sách liên kết (LinkedList<Customer>), Danh sách mảng động (ArrayList<Object>) hay các nút cây (TreeNode), các đối tượng gen sẽ được cấp phát ngẫu nhiên trên bộ nhớ Heap. Điều này dẫn đến hiện tượng **Phân mảnh bộ nhớ (Memory Fragmentation)**.

CẤU TRÚC DỰA TRÊN CON TRỎ (Gây Cache Miss):
Heap Memory: [Obj_A] ---> (Pointer) ---> [Obj_B] ---> (Pointer) ---> [Obj_C]
CPU Cache: [Miss!] [Miss!] [Miss!]

CẤU TRÚC MẢNG 1 CHIỀU TUẦN TỰ (Tối ưu Cache Line Prefetching):
RAM/L1-L3: [Gen_0 | Gen_1 | Gen_2 | Gen_3 | Gen_4 | Gen_5 | Gen_6 | Gen_7]
CPU Cache: [========== Loaded contiguous Cache Line (64 Bytes) ==========]

Phân tích ảnh hưởng tới kiến trúc vi xử lý (Microarchitecture Impact):

1. **Trạng thái Cache Miss liên tục:** Khi CPU thực hiện các thao tác di truyền (cắt chuỗi, đổi chỗ gen, tính tổng khoảng cách) với tần suất hàng triệu lần mỗi giây, việc truy xuất qua các con trỏ khiến CPU liên tục rơi vào trạng thái *Cache Miss* ở các tầng bộ nhớ đệm L1, L2 và L3. CPU buộc phải dừng chu kỳ xử lý (Stall Cycles) để nạp lại dữ liệu từ bộ nhớ RAM chính với độ trễ cao hơn gấp vài trăm lần ($100 \div 200$ ns so với $1 \div 2$ ns của L1 Cache).
2. **Nguyên lý tập trung không gian (Spatial Locality) & Nạp trước bộ nhớ đệm (Cache Line Prefetching):** Bằng việc chuyển đổi toàn bộ cấu trúc dữ liệu nhiễm sắc thể sang **Mã hóa mảng 1 chiều số nguyên tuần tự (Contiguous Primitive 1D Array)**, tất cả các phần tử gen được lưu trữ trên các ô nhớ liên tiếp nhau. Khi CPU truy xuất gen i, cơ chế Hardware Prefetcher sẽ tự động nạp trọn vẹn một Dòng bộ nhớ đệm (Cache Line - thường là 64 Bytes) chứa các gen kế tiếp \(i+1, i+2, \dots\) vào L1 Cache.

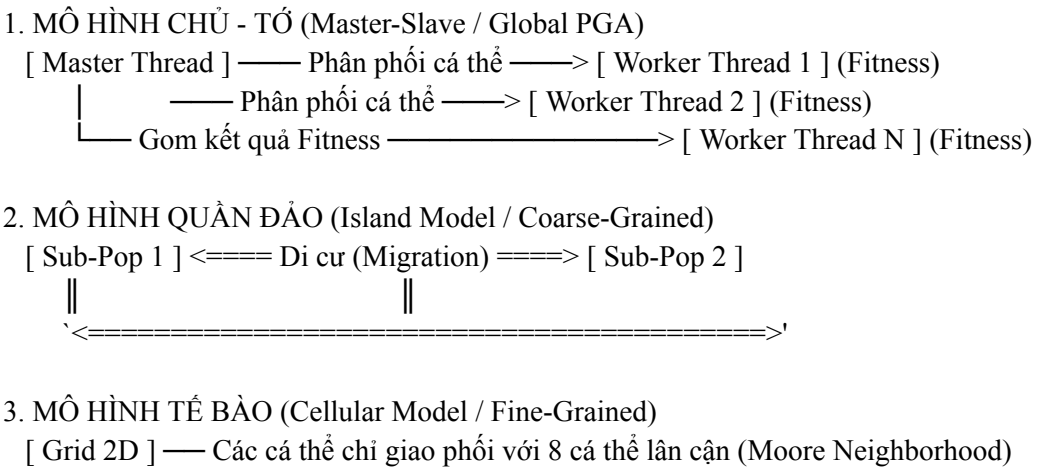
Kết quả tối ưu hóa:

- Tốc độ đọc/ghi mảng trong các vòng lặp di truyền đạt độ phức tạp truy xuất bộ nhớ tức thì $O(1)$.
- Triệt tiêu hoàn toàn chi phí tạo mới đối tượng (Object Overhead) và giảm thiểu áp lực lên bộ thu gom rác (Garbage Collector - GC), ngăn ngừa hiện tượng ứng dụng Backend bị khựng (GC Pauses) trong quá trình tính toán thời gian thực.

1.6.2. Các mô hình tính toán song song cho Thuật toán Di truyền (Parallel GA Paradigms)

Để đáp ứng khắt khe ngân sách thời gian xử lý lô Batching ($b \leq 3 \div 5$ giây), giải pháp duy nhất là khai thác triệt để kiến trúc vi xử lý đa nhân (Multi-core CPUs) thông qua các mô hình tính toán song

song. Trong lý thuyết tính toán song song dành cho Thuật toán Di truyền (Parallel Genetic Algorithms - PGA), có 3 mô hình phân phân phối cốt lõi:



1.6.2.1. Mô hình Quần đảo (Island Model / Coarse-Grained PGA)

Quần thể tổng thể được chia nhỏ thành M tiểu quần thể (Islands) độc lập hoạt động trên các thread khác nhau. Định kỳ sau một số thế hệ (Migration Interval), các cá thể ưu tú nhất sẽ "di cư" (Migrate) sang các quần đảo khác để trao đổi nguồn gen.

- *Nhược điểm:* Khó kiểm soát tốc độ hội tụ và dễ phát sinh chi phí đồng bộ hóa (Synchronization Overhead) khi quy mô lô đơn hàng biến động.

1.6.2.2. Mô hình Tế bào (Cellular Model / Fine-Grained PGA)

Các cá thể được sắp xếp trên một lưới không gian $2D$. Mỗi cá thể chỉ thực hiện lai ghép và đột biến với các cá thể thuộc vùng lân cận hẹp xung quanh nó.

- *Nhược điểm:* Phù hợp với hạ tầng phần cứng mảng SIMD/GPU hơn là kiến trúc máy chủ Backend CPU bộ nhớ chia sẻ.

1.6.2.3. Mô hình Chủ - Tớ Đa luồng Bộ nhớ Chia sẻ (Master-Slave / Global Parallel GA)

Đề tài đề xuất áp dụng **Mô hình Chủ - Tớ Bộ nhớ Chia sẻ (Shared-Memory Master-Slave Paradigm)**:

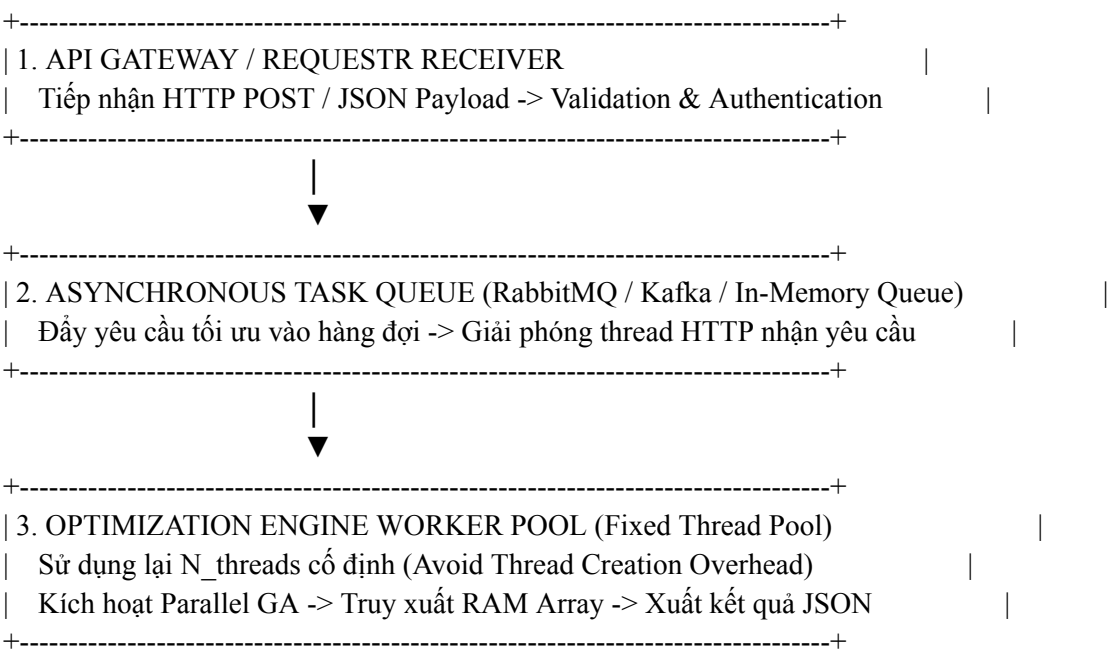
- **Tiến trình Chủ (Master Thread):** Chịu trách nhiệm quản lý quần thể tập trung, thực hiện các toán tử chọn lọc tự nhiên (Tournament Selection), quản lý quần thể phụ và kiểm tra điều kiện dừng.
- **Các Tiến trình Tớ (Slave / Worker Threads):** Nhận các phân đoạn cá thể từ Master để tính toán song song Ma trận khoảng cách và **Hàm thích nghi (Fitness Evaluation)** – vốn là tác vụ chiếm tới $80\% \div 90\%$ tổng thời gian tính toán của toàn hệ thống.

Bảng 1.3. Phân tích so sánh các mô hình song song hóa Thuật toán Di truyền

Tiêu chí so sánh	Mô hình Chủ - Tớ (Master-Slave)	Mô hình Quần đảo (Island Model)	Mô hình Tế bào (Cellular Model)
Cấu trúc Quần thể	Một quần thể duy nhất quản lý tập trung.	Phân rã thành nhiều tiểu quần thể.	Quần thể bố trí dạng lưới $2D$ không gian.
Thao tác Song song hóa	Song song hóa đánh giá Fitness.	Tiến hóa song song trên các đảo.	Đột biến/Lai ghép song song lân cận.
Mức độ Đa dạng Gen	Tối ưu toàn cục theo chu kỳ thế hệ.	Đa dạng cao nhờ cơ chế Di cư.	Đa dạng cực cao; hội tụ chậm.
Độ phức tạp Cài đặt Backend	**Trung bình; Tương thích hoàn hảo Java Thread Pool.**	Phức tạp (Cần quản lý luồng di cư).	Cực kỳ phức tạp (Yêu cầu kỹ thuật GPU/CUDA).
Phù hợp xử lý Batching	**Rất cao (Đáp ứng phản hồi $<$ 3 giây).**	Trung bình.	Thấp đối với CPU đa nhân chuẩn.

1.6.3. Tối ưu hóa thời gian thực thi (Real-time Latency) và Đóng gói Microservice

Nhằm đưa Lõi tối ưu hóa vào môi trường sản xuất (Production Environment), Engine Backend được đóng gói dưới dạng một **Microservice đọc lập (Optimization Engine Microservice)** theo kiến trúc hướng sự kiện (Event-driven Architecture):



Các kỹ thuật tối ưu hóa độ trễ phản hồi (Latency Optimization Techniques):

1. **Sử dụng Pool luồng cố định (Fixed Thread Pool Management):** Việc khởi tạo mới một Thread trong hệ điều hành tốn chi phí khoảng 1 MB bộ nhớ Stack và hàng ngàn chu kỳ CPU. Engine thiết lập sẵn một Pool luồng cố định khớp đúng với số nhân thực tế của máy chủ ($N_{threads} = N_{CPU_cores}$) để tái sử dụng luồng liên tục, giảm thiểu thời gian chờ (Thread Overhead = 0).
2. **Giao tiếp Bất đồng bộ (Asynchronous Non-blocking I/O):** Kết nối giữa API Gateway và Optimization Engine được xử lý bất đồng bộ qua hàng đợi thông điệp (Message Queue), giúp hệ thống không bị nghẽn (non-blocking) khi số lượng lô đơn hàng gửi lên tăng đột biến.

1.7. Ứng dụng ma trận khoảng cách và tích hợp dịch vụ bản đồ địa lý

1.7.1. Bản chất Khoảng cách Euclid vs. Khoảng cách Thực tế trên Mạng lưới Đường bộ

Trong các nghiên cứu lý thuyết cổ điển, khoảng cách giữa hai điểm giao hàng $i(x_i, y_i)$ và $j(x_j, y_j)$ thường được đơn giản hóa bằng khoảng cách Euclid hình học:

$$d_{ij}^{Euclid} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

Tuy nhiên, trong hạ tầng giao thông đô thị thực tế, việc áp dụng khoảng cách Euclid dẫn đến **sai lệch nghiêm trọng** so với thực địa:

- **Hình thái mạng lưới giao thông:** Phương tiện di chuyển theo mạng lưới đường ô bàn cờ (Khoảng cách Manhattan $d_{ij}^{Manhattan} = |x_i - x_j| + |y_i - y_j|$) hoặc theo các tuyến đường cong phụ thuộc địa hình.
- **Rào cản địa lý và Quy tắc Giao thông:** Sự xuất hiện của sông ngòi (phụ thuộc vào vị trí cầu bắc qua), đường một chiều (One-way streets), rào chắn dải phân cách và cấm rẽ trái/quay đầu khiến đường di chuyển từ i đến j khác hoàn toàn đường từ j về i.
- **Tính Bất đối xứng (Asymmetry):** Chi phí và khoảng cách thực tế mang tính bất đối xứng rõ rệt:
- $c_{ij} \neq c_{ji}$(đồ$thị có hướng G = (V, A)$)
-

Khoảng cách Euclid (Chim bay): d_ij = 3.2 km (Tuyệt đối đối xứng)
(Khách hàng i) . - - - - - - - - - - - - . (Khách hàng j)

Chi phí Thực tế Đô thị (Mạng lưới đường bộ có hướng):
(Khách hàng i) —— Đường 1 chiều ——> [Cầu] ——> (Khách hàng j) : c_ij = 5.8 km
(Khách hàng j) —— Tuyến vòng tránh ————————————> (Khách hàng i) : c_ji = 8.1 km

1.7.2. Cơ chế Bộ nhớ đệm Ma trận Chi phí (Cost Matrix Caching) và Kỹ thuật Pre-computation

Khi vận hành thực tế, việc gọi liên tục các dịch vụ bản đồ (Google Maps Distance Matrix API, OpenStreetMap Service) để lấy khoảng cách giữa từng cặp điểm trong quá trình chạy hàng ngàn thế hệ di truyền sẽ gây ra hai tai họa kỹ thuật:

1. **Độ trễ mạng cực lớn (Network Latency):** Mỗi yêu cầu HTTP API mất từ $100 \div 300$ ms. Nếu thuật toán gọi API trực tiếp trong hàm Fitness, thời gian chạy một thế hệ sẽ kéo dài hàng giờ!
2. **Chi phí tài chính (API Billing Cost):** Tốn hàng ngàn USD chi phí gọi API cho các nhà cung cấp bản đồ thương mại.

Giải pháp Kiến trúc Backend – Kỹ thuật Pre-computation & In-Memory Caching:

- **Bước 1 (Pre-computation):** Tại mốc chốt Batch t_k, Lõi Backend trích xuất danh sách n điểm đơn hàng + Kho 0. Hệ thống thực hiện duy nhất 1 **lượt gọi theo lô (Batch Call)** tới Map Engine để lấy trọn vẹn Ma trận chi phí $(n + 1) \times (n + 1)$.
- **Bước 2 (In-Memory Caching):** Toàn bộ ma trận chi phí c_{ij} và thời gian di chuyển τ_{ij} được nạp vào một **Mảng hai chiều tĩnh trong bộ nhớ RAM (double[][] costMatrix)**.
- **Bước 3 (Look-up $O(1)$):** Trong suốt quá trình Thuật toán Di truyền (GA) vận hành qua hàng ngàn thế hệ, hàm đánh giá Fitness chỉ truy xuất giá trị c_{ij} bằng chỉ mục mảng costMatrix[i][j] với độ phức tạp $O(1)$ và thời gian truy xuất chỉ $1 \div 2$ ns.

1.7.3. Thuật toán Dijkstra/A* Song song Hai chiều (Parallel Bidirectional Shortest Path)

Khi doanh nghiệp tự vận hành hạ tầng bản đồ nội bộ (Private GIS Engine dựa trên dữ liệu OpenStreetMap - OSM), việc tính toán đường đi ngắn nhất giữa các nút giao thông đòi hỏi các thuật toán tìm kiếm đường đi hiệu năng cao. Trong luận án Tiến sĩ của **Gintaras Vaira (2014)** tại Đại học Vilnius, tác giả đã chứng minh việc nâng cao tốc độ tính toán ma trận đường đi bằng **Thuật toán Dijkstra Song song Hai chiều (Parallel Bidirectional Dijkstra's Shortest Path Algorithm)**.

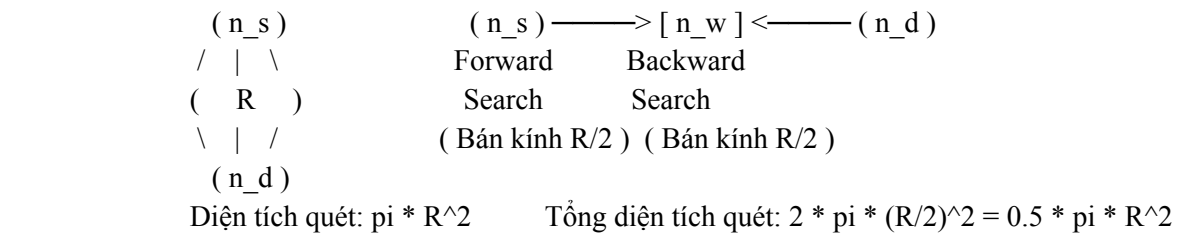
1.7.3.1. Nguyên lý thuật toán Dijkstra Hai chiều (Bidirectional Search)

Thay vì chỉ tìm kiếm một chiều từ đỉnh gốc n_s lan rộng ra toàn đồ thị, thuật toán kích hoạt đồng thời hai tiến trình tìm kiếm:

1. **Tìm kiếm Tiến (Forward Search):** Xuất phát từ đỉnh gốc n_s hướng tới đỉnh đích n_d với hàng đợi ưu tiên Q_S.
2. **Tìm kiếm Lùi (Backward Search):** Xuất phát ngược từ đỉnh đích n_d về đỉnh gốc n_s với hàng đợi ưu tiên Q_D.

Hai hướng tìm kiếm sẽ "gặp nhau" tại một đỉnh trung gian n_w ở khoảng giữa không gian đồ thị, giúp giảm không gian đỉnh cần quét từ một hình tròn bán kính R (diện tích πR^2) xuống còn hai hình tròn nhỏ bán kính $R/2$ (tổng diện tích $2 \times \pi(R/2)^2 = \frac{1}{2}\pi R^2$), tiết kiệm 50% số đỉnh phải xử lý.

Dijkstra Chuẩn 1 chiều: Dijkstra Song song 2 chiều:



Diện tích quét: pi * R^2 Tổng diện tích quét: 2 * pi * (R/2)^2 = 0.5 * pi * R^2

1.7.3.2. Mã giả Thuật toán Dijkstra Song song Hai chiều bộ nhớ chia sẻ

```
procedure ParallelBidirectionalDijkstra(Graph G=(N, E), node_start, node_dest):
   // Khởi tạo các mảng khoảng cách và tập hợp đỉnh đã duyệt trong bộ nhớ chia sẻ
   dist_F[node_start] = 0; dist_B[node_dest] = 0
   visited_F = EMPTY_SET; visited_B = EMPTY_SET
   candidate_answers = EMPTY_SET
   finish_phase = false

   // Khởi tạo hai Thread độc lập chạy song song trên CPU
        THREAD_1 = SpawnThread(ForwardSearch, G, node_start, visited_F, visited_B, candidate_answers)
        THREAD_2 = SpawnThread(BackwardSearch, G, node_dest, visited_B, visited_F, candidate_answers)

   JoinThread(THREAD_1)
   JoinThread(THREAD_2)

   // Trích xuất đường đi ngắn nhất tuyệt đối từ tập lời giải ứng viên Z
   return ExtractShortestPath(candidate_answers)

procedure ForwardSearch(G, node_start, visited_self, visited_opposite, Z):
   Q_S.insert(node_start, 0)
   while Q_S is not empty:
      u = Q_S.extractMin()
      visited_self.add(u)

      // Điều kiện dừng: Khi phát hiện đỉnh u đã được thread đối diện duyệt qua
      if u in visited_opposite:
         finish_phase = true
         Z.addAnswerPath(u, dist_F[u] + dist_B[u])
         break

      if not finish_phase:
         for each neighbor v of u:
```

```
if dist_F[v] > dist_F[u] + weight(u, v):
    dist_F[v] = dist_F[u] + weight(u, v)
    Q_S.decreaseKey(v, dist_F[v])
```

1.7.3.3. Đánh giá hiệu năng thực nghiệm

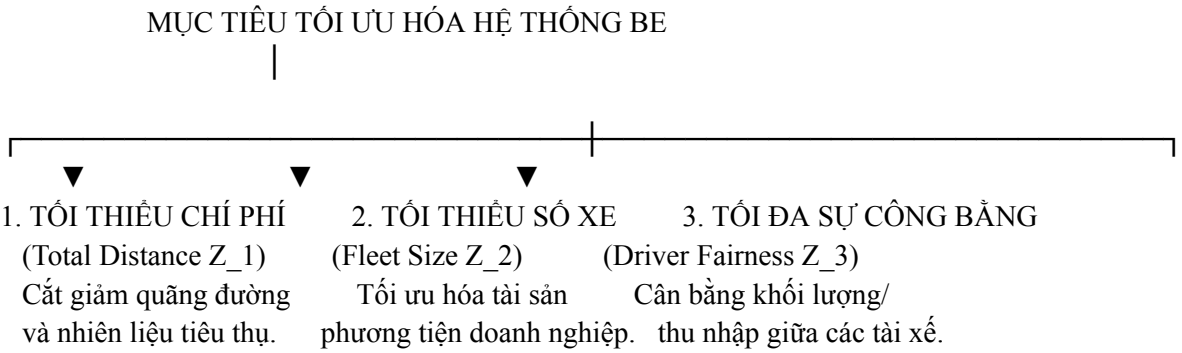
Nghiên cứu của Vaira (2014) trên dữ liệu mạng lưới đường bộ OpenStreetMap thực tế đã khẳng định:

- Thuật toán Dijkstra Hai chiều tuần tự nhanh hơn 2.0 **lần** so với Dijkstra chuẩn.
- Thuật toán Dijkstra **Song song Hai chiều (Parallel Bidirectional)** chạy trên CPU 2 nhân đạt hệ số tăng tốc $S_p \approx 2.9$ **lần** so với Dijkstra chuẩn, giúp việc khởi tạo ma trận chi phí cho Engine Backend diễn ra trong mili-giây.

1.8. Mô hình hóa mục tiêu đa chiều và đánh giá tính ổn định của lõi tối ưu hóa

1.8.1. Cân bằng giữa Chi phí Vận chuyển, Tải trọng Xe và Sự Công bằng trong Điều phối

Trong môi trường điều phối thực tế của các doanh nghiệp logistics và dịch vụ gọi xe/giao hàng công nghệ, một phương án phân công lộ trình hoàn hảo không thể chỉ chú trọng đơn mục tiêu là cực tiểu hóa tổng quãng đường di chuyển. Hệ thống Backend phải giải quyết bài toán mâu thuẫn giữa 3 hàm mục tiêu:



1. **Hàm mục tiêu 1 (Chi phí Vận chuyển - Z_1):** Cực tiểu hóa tổng quãng đường di chuyển của toàn đội xe nhằm tiết kiệm nhiên liệu và giảm lượng khí thải:
2. $Z_1 = \sum_{k=1}^{K} \sum_{i \in V} \sum_{j \in V} c_{ij} x_{ijk}$
3.
4. **Hàm mục tiêu 2 (Quy mô Đội xe - Z_2):** Cực tiểu hóa tổng số lượng phương tiện thực tế cần điều động K_{used} để cắt giảm chi phí cố định (khấu hao xe, chi phí tài xế):
5. $Z_2 = \sum_{k=1}^{K} \left(\max_{j \in C, i \in V} x_{0jk} \right)$

6.

7. **Hàm mục tiêu 3 (Chỉ số Công bằng Tài xế - Z_3 - Driver Fairness):** Theo nghiên cứu của **Lâm Thái Yến Nhi (2026)**, việc gán dồn quá nhiều đơn hàng cho một vài tài xế trong khi các tài xế khác bị bỏ rỗng sẽ gây ra sự bất mãn và gia tăng tỷ lệ rời bỏ hệ thống. Hệ thống đo lường sự công bằng bằng **Phương sai khối lượng công việc (Workload Variance)** giữa các tuyến xe:

8. $Z_3 = \frac{1}{K}\sum_{k=1}^{K}\left(Q_k - \bar{Q}\right)^2$

9. *(Trong đó $Q_k = \sum_{i \in Route_k} q_i$ là tổng tải trọng xe k phụ trách và $\bar{Q} = \frac{1}{K}\sum Q_k$ là tải trọng trung bình).*

Giải pháp tóm gọn dạng Hàm Thích nghi Tổng hợp (Scalarized Fitness Function): Để xử lý đa mục tiêu trong một thuật toán GA đơn quần thể mà vẫn giữ được tốc độ phản hồi thời gian thực, Engine Backend đóng gói thành hàm thích nghi trọng số kết hợp hàm phạt vi phạm:

\[\text{Fitness}(X) = w_1 \cdot Z_1 + w_2 \cdot (Z_2 \cdot C_{\text{fixed}}) + w_3 \cdot Z_3 + \alpha \cdot \text{Penalty}{\text{Capacity}}(X) + \beta \cdot \text{Penalty}{\text{TimeWindow}}(X)\]

Trong đó w_1, w_2, w_3 là các hệ số trọng số do doanh nghiệp cấu hình tùy theo chiến lược kinh doanh tại từng thời điểm.

1.8.2. Đánh giá tính ổn định qua các lần chạy hạt giống ngẫu nhiên (Random Seed Stability)

Do Thuật toán Di truyền (GA) thuộc lớp **Metaheuristic ngẫu nhiên (Stochastic Metaheuristics)**, việc chạy thuật toán trên cùng một tập dữ liệu đầu vào với các Hạt giống ngẫu nhiên (Random Seeds) khác nhau có thể cho ra các nghiệm với chất lượng chênh lệch.

Đối với một Lõi phần mềm Backend chuẩn công nghiệp, tính ổn định (Stability & Robustness) là chỉ tiêu đánh giá sống còn. Đề tài thiết lập quy trình kiểm thử lặp lại M **lần độc lập (**$M = 10 \div 30$ **lượt)** trên mỗi thể hiện bài toán CVRPLib với các seed ngẫu nhiên khác nhau.

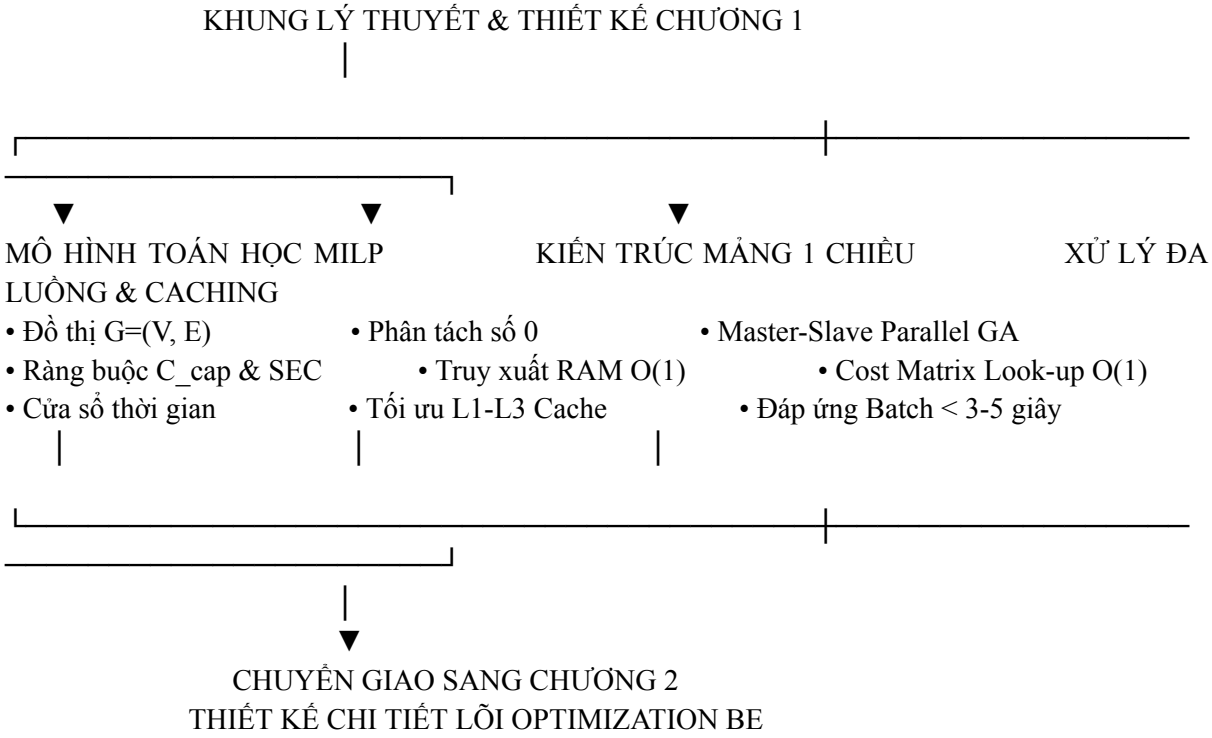
Bảng 1.4. Bộ chỉ số thống kê đo lường tính ổn định của Lõi Optimization Backend

Chỉ số thống kê	Công thức toán học	Ý nghĩa thực tiễn đối với Phần mềm Backend
Giá trị Tốt nhất (Best)	$Z_{best} = \min_{m=1..M} Z_m$	Năng lực bứt phá tìm kiếm nghiệm tiệm cận tối ưu toàn cục.
Giá trị Trung bình (Mean)	$\bar{Z} = \frac{1}{M}\sum_{m=1}^{M} Z_m$	Chất lượng kỳ vọng trung bình của phương án điều phối khi vận hành tự động.

Độ lệch chuẩn (Std Dev)	$\sigma_Z = \sqrt{\frac{1}{M-1}\sum_{m=1}^{M}(Z_m - \bar{Z})^2}$	**Thước đo độ ổn định:** σ_Z càng nhỏ chứng tỏ Engine càng ít bị biến động bởi yếu tố ngẫu nhiên.
Hệ số Biến thiên (CV %)	$CV(\%) = \left(\frac{\sigma_Z}{\bar{Z}}\right) \times 100\%$	Cho biết mức độ phân tán tương đối của kết quả qua các lần khởi tạo hạt giống.

1.8.3. Khái quát hóa khung thử nghiệm và Kết luận Định hướng cho Chương 2

Toàn bộ các phân tích lý thuyết, mô hình toán học, kiến trúc đa luồng và phương pháp đo lường được xây dựng trong **Chương 1** đã thiết lập nền tảng lý luận vững chắc cho đồ án:



1. **Sự tất yếu của Lõi Optimization Backend:** Phân tích bùng nổ tổ hợp NP-Hard khẳng định các phương pháp chính xác bị sụp đổ khi $n \geq 30$, trong khi Heuristic cổ điển bị kẹt trong bẫy Tối ưu cục bộ. Thuật toán Di truyền (GA) được lựa chọn là lõi thuật toán chính.
2. **Định hướng Kỹ thuật Phần mềm:** Đề tài tập trung thiết kế cấu trúc dữ liệu **Mã hóa mảng 1 chiều phân tách số 0**, tối ưu hóa bộ nhớ đệm CPU (Cache Locality), ứng dụng **Mô hình song song đa luồng Master-Slave** và cơ chế Pre-computation Ma trận chi phí để xây dựng Microservice Tối ưu hóa phản hồi trong $3 \div 5$ giây.
3. **Mô hình hóa thực tế:** Tích hợp ràng buộc thời gian hẹp, khoảng cách giao thông bất đối xứng thực tế và mô hình đa mục tiêu kết hợp công bằng tài xế.

Từ các định hướng cốt lõi trên, **Chương 2** sẽ đi sâu vào **Cài đặt Chi tiết Thuật toán Di truyền, Cấu trúc Dữ liệu Mảng 1 chiều và Kiến trúc Phần mềm Lõi Backend**.

TÀI LIỆU THAM KHẢO

Nguyễn Việt Hân, *"Thuật toán di truyền song song giải bài toán VRP với hạn chế thời gian"*, Luận văn Thạc sĩ Khoa học Máy tính, Đại học Bách Khoa Hà Nội, 2009.

Gintaras Vaira, *"Genetic Algorithm for Vehicle Routing Problem"*, Doctoral Dissertation, Vilnius University, Institute of Mathematics and Informatics, Lithuania, 2014.

Lâm Thái Yến Nhi, *"Tiếp cận bằng thuật toán NSGA-II cho bài toán điều phối gọi xe đa mục tiêu theo cơ chế Batching có ràng buộc cửa sổ thời gian"*, Báo cáo Đồ án Chuyên ngành, Trường Đại học Sài Gòn, 2026.

Cao Ngọc Ánh, Trần Bích Thảo, *"Ứng dụng giải thuật Tabu Search trong giải bài toán định tuyến xe"*, Tạp chí Khoa học & Công nghệ, Số 38, tr. 37-43, 2023.

Thái Quốc Bảo, *"Nghiên cứu thuật toán Di truyền giải bài toán định tuyến xe trong giao vận đô thị"*, Bản thảo Đồ án Chuyên ngành, Trường Đại học Sài Gòn, 2026.

B. M. Baker and M. A. Ayechew, *"A genetic algorithm for the vehicle routing problem"*, *Computers & Operations Research*, vol. 30, no. 5, pp. 787-800, 2003.

S. Uchoa, E. Pecin, A. Pessoa, M. Poggi, T. Vidal, and A. Subramanian, *"New benchmark instances for the Capacitated Vehicle Routing Problem"*, *European Journal of Operational Research*, vol. 257, no. 3, pp. 845-858, 2017.

M. M. Solomon, *"Algorithms for the Vehicle Routing and Scheduling Problems with Time Window Constraints"*, *Operations Research*, vol. 35, no. 2, pp. 254-265, 1987.

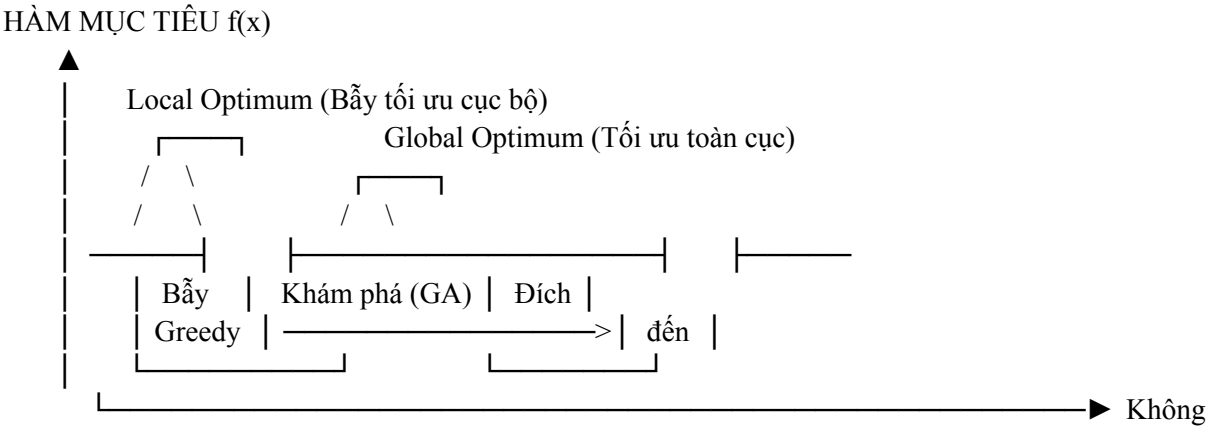
CHƯƠNG 2. THUẬT TOÁN DI TRUYỀN VÀ GIẢI PHÁP ĐỀ XUẤT (THIẾT KẾ LÕI OPTIMIZATION BACKEND ENGINE)

2.1. Cơ sở lý thuyết và nền tảng thiết kế giải pháp tối ưu hóa

2.1.1. Luận điểm chuyển dịch từ Heuristic cổ điển sang Metaheuristic (Genetic Algorithm)

Như đã chứng minh ở Chương 1, các phương pháp kinh nghiệm cổ điển (Classical Heuristics) như Thuật toán Tiết kiệm (Savings Algorithm) của Clarke & Wright hay Thuật toán Quét góc (Sweep Algorithm) xây dựng phương án phân công dựa trên tư duy "tham lam" (Greedy Approach). Nguyên lý vận hành của nhóm thuật toán này là liên tục thực hiện các di chuyển chuyển dịch lân cận làm giảm trực tiếp chi phí hàm mục tiêu $f(x') < f(x)$.

Tuy nhiên, trong không gian trạng thái tổ hợp phi tuyến, không lồi và gập ghềnh của Bài toán Định tuyến Xe có Ràng buộc Sức chứa (CVRP), bề mặt hàm mục tiêu chứa vô số các "thung lũng" tối ưu cục bộ (Local Optima). Khi thuật toán rơi vào một thung lũng mà mọi nghiệm lân cận $x' \in N(x)$ đều có chi phí cao hơn điểm hiện tại x, thuật toán bị mắc kẹt hoàn toàn tại điểm này. Việc thiếu vắng cơ chế chấp nhận tạm thời các bước di chuyển xấu hơn khiến các Heuristics cổ điển không thể "leo dốc" (hill-climbing) để vượt qua rào cản không gian nhằm tìm kiếm các vùng không gian hứa hẹn khác.



Để khắc phục dứt điểm nhược điểm chí mạng này, giải pháp của đề tài thực hiện sự chuyển dịch nền tảng sang các **Siêu thuật toán kinh nghiệm (Metaheuristics)** – tiêu biểu là **Thuật toán Di truyền (Genetic Algorithm - GA)**. Metaheuristics không phải là một thuật toán giải trực tiếp một bài toán cụ thể, mà là một khung lý thuyết tìm kiếm thông minh (Search Framework) sở hữu khả năng cân bằng giữa hai chiến lược nền tảng:

1. **Khai thác (Intensification / Exploitation):** Đào sâu tìm kiếm trong vùng lân cận của các cá thể ưu tú để tinh chỉnh và tối ưu hóa chi tiết lộ trình.

2. **Khám phá (Diversification / Exploration):** Mở rộng vùng tìm kiếm sang các lãnh thổ hoàn toàn mới trong không gian trạng thái nhằm duy trì tính đa dạng di truyền và chủ động thoát khỏi bẫy tối ưu cục bộ.

Thuật toán Di truyền đạt được sự cân bằng này nhờ cơ chế tiến hóa quần thể kết hợp với tính chất ngẫu nhiên có định hướng của các toán tử sinh học (Lai ghép, Đột biến).

2.1.2. Cơ sở lý thuyết của Thuật toán Di truyền (Genetic Algorithm - GA) trong Tối ưu hóa Tổ hợp

Thuật toán Di truyền do John Holland khởi xướng vào thập niên 1970 và được David E. Goldberg hoàn thiện lý thuyết nền tảng, mô phỏng lại nguyên lý tiến hóa tự nhiên và chọn lọc sinh học theo thuyết Darwin: *"Kẻ mạnh nhất sẽ tồn tại"* (Survival of the Fittest). Trong tối ưu hóa máy tính, các khái niệm sinh học được ánh xạ toán học một cách chặt chẽ:

Bảng 2.1. Ánh xạ giữa Sinh học tiến hóa và Thuật toán Di truyền trong CVRP

Khái niệm Sinh học	Ánh xạ trong Khoa học Máy tính	Ý nghĩa kỹ thuật trong Bài toán CVRP
Cá thể (Individual)	Một phương án giải quyết bài toán ($x \in S$).	Một tập hợp bao gồm các tuyến đường hoàn chỉnh cho đội xe K.
Nhiễm sắc thể (Chromosome)	Cấu trúc dữ liệu mã hóa thông tin cá thể.	Mảng số nguyên lưu trữ chuỗi ID khách hàng và vách ngăn.
Gen (Gene)	Đơn vị thông tin cơ bản trên nhiễm sắc thể.	Mã định danh (ID) của một điểm giao hàng hoặc trạm gốc 0.
Quần thể (Population)	Tập hợp P cá thể tại thế hệ t (\(P_t = {x_1, \dots, x_P}\)).	Không gian chứa hàng trăm phương án phân công chạy song song.
Độ thích nghi (Fitness)	Khả năng sống sót và thích nghi môi trường.	Giá trị đo lường dựa trên tổng quãng đường Z và hệ số phạt.
Chọn lọc (Selection)	Quy luật chọn lọc tự nhiên.	Chọn các cá thể có Fitness cao làm Bố/Mẹ để truyền gen.
Lai ghép (Crossover)	Tái tổ hợp di truyền giữa hai cá thể Bố/Mẹ.	Lắp ghép các phân đoạn lộ trình tốt từ Bố và Mẹ.
Đột biến (Mutation)	Biến đổi ngẫu nhiên cấu trúc gen.	Xáo trộn ngẫu nhiên thứ tự giao hàng để tạo gen mới.

2.1.2.1. Vòng đời tiến hóa tổng quát của Thuật toán Di truyền

Thứ tự thực thi tuần tự của một thuật toán Di truyền chuẩn mực được mô hình hóa qua các bước thuật toán chính:

```
# Mô hình thuật toán GA chuẩn
procedure GeneticAlgorithm_Standard():
   t = 0
   P[t] = InitializePopulation(P_size)     # 1. Khởi tạo quần thể ban đầu
   EvaluateFitness(P[t])                   # 2. Đánh giá độ thích nghi

   while not TerminationCriterion():      # Vòng lặp tiến hóa qua các thế hệ
      Parents = SelectParents(P[t])      # 3. Chọn lọc Bố/Mẹ
      Offspring = Crossover(Parents)     # 4. Tái tổ hợp Lai ghép
      MutatedOffspring = Mutate(Offspring)# 5. Đột biến di truyền
      EvaluateFitness(MutatedOffspring)  # 6. Đánh giá cá thể con
      P[t+1] = Recombination(P[t], MutatedOffspring) # 7. Sắp xếp & Thay thế
      t = t + 1

   return GetBestIndividual(P[t])         # Trả về cá thể tối ưu toàn cục
```

2.1.2.2. Các phương trình toán học chi tiết trong vòng đời GA

1. Hàm thích nghi (Fitness Evaluation)

Khác với các bài toán tìm giá trị cực đại, bài toán CVRP hướng tới cực tiểu hóa tổng chi phí quãng đường Z. Để áp dụng nguyên tắc chọn lọc tự nhiên (cá thể có chỉ số Fitness càng lớn càng tốt), hàm thích nghi $F(x)$ của cá thể x được nghịch đảo toán học hoặc chuyển đổi tuyến tính:

\[F(x) = \frac{1}{Z(x) + \alpha \cdot \text{Penalty}_{\text{Capacity}}(x) + \beta \cdot \text{Penalty}_{\text{TimeWindow}}(x) + \epsilon}\]

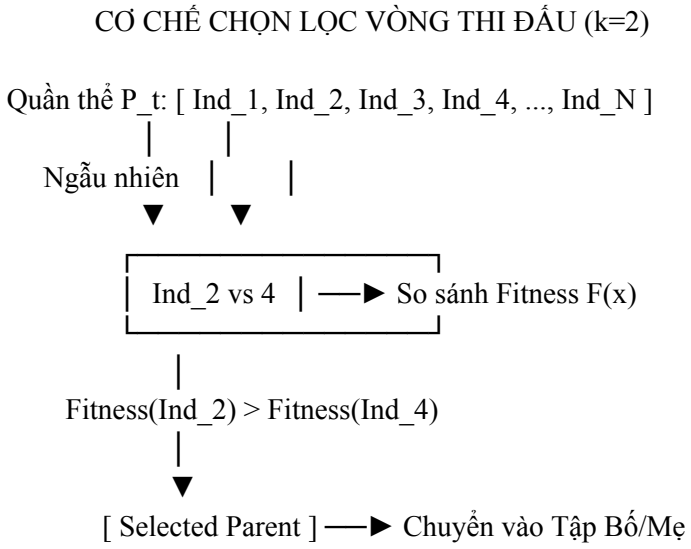
Trong đó:

- $Z(x)$: Tổng quãng đường di chuyển của tất cả các xe trong cá thể x.
- \(\text{Penalty}_{\text{Capacity}}(x)\)*: Tổng khối lượng quá tải kỹ thuật vi phạm sức chở* \(C_{\text{cap}}\).
- $Penalty_{TimeWindow}(x)$: Tổng thời gian vi phạm cửa sổ phục vụ $[e_i, l_i]$.
- α, β: Các hệ số phạt (Penalty Multipliers) được điều chỉnh linh hoạt.
- $\epsilon > 0$: Số hạng cực nhỏ tránh lỗi chia cho 0.

2. Cơ chế Chọn lọc theo Vòng thi đấu (Tournament Selection)

Nhằm duy trì **Áp lực chọn lọc (Selective Pressure)** phù hợp và tránh hiện tượng cá thể quá mạnh chiếm lĩnh sớm quần thể, đề tài áp dụng cơ chế Chọn lọc Vòng thi đấu (k-Tournament Selection):

- Rút ngẫu nhiên k cá thể (thường chọn $k = 2$ hoặc $k = 5$) từ quần thể P_t.
- So sánh chỉ số Fitness giữa k cá thể này. Cá thể có Fitness cao nhất chiến thắng và được đưa vào tập Bố/Mẹ (Parent Pool).
- Xác suất cá thể xếp thứ r trong Tournament được chọn làm Bố/Mẹ tuân theo phân phối:
- $P(Select) = p \cdot (1 - p)^{r-1}$
-



3. Tái tổ hợp Lai ghép (Crossover) và Đột biến (Mutation)

- **Hệ số Lai ghép (p_c - Crossover Rate):** Quy định tỷ lệ cặp Bố/Mẹ thực hiện trao đổi chuỗi gen (thường chọn $p_c = 0.8 \div 0.9$).
- **Hệ số Đột biến (p_m - Mutation Rate):** Quy định xác suất biến đổi gen trên cá thể con (thường chọn $p_m = 0.01 \div 0.15$). Đột biến đóng vai trò là "nguồn biến dị di truyền", tiêm các chuỗi gen mới vào quần thể để ngăn chặn trạng thái thoái hóa đồng nhất.

2.1.3. Mô hình Tối ưu hóa Backend (Backend Optimization Engine) và Kiến trúc Đa luồng (Parallel GA)

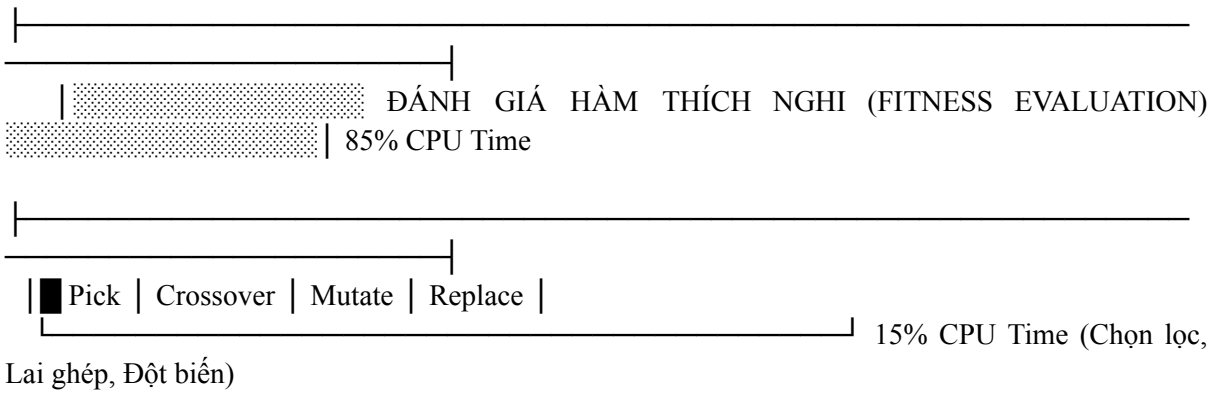
Trong kỹ thuật phần mềm hiện đại, việc nghiên cứu Thuật toán Di truyền không thể chỉ dừng lại ở các kịch bản chạy thử nghiệm đơn luồng trên máy tính cá nhân. Đề tài định hướng đóng gói thuật toán thành một **Optimization Backend Engine (Microservice)** độc lập, có khả năng tích hợp qua RESTful API/JSON để phục vụ các hệ thống điều phối logistics thương mại.

2.1.3.1. Thách thức độ trễ tính toán và Yêu cầu Song song hóa

Quá trình tiến hóa của GA đòi hỏi phải lặp qua hàng trăm đến hàng ngàn thế hệ ($N_{gen} = 500 \div 1000$) trên một quần thể gồm $P = 100 \div 500$ cá thể. Trong mỗi thế hệ, việc đánh giá hàm thích nghi (Fitness Evaluation) yêu cầu duyệt mảng, tính toán tổng khoảng cách Euclid/Map API và kiểm tra chuỗi ràng buộc cho từng cá thể.

Phân tích định lượng chi phí thời gian thực thi (Computational Profiling) khẳng định: **Bước đánh giá Hàm thích nghi chiếm từ** $80\%đến n90\%$ **tổng thời gian tính toán của toàn hệ thống**. Nếu thực thi tuần tự trên một luồng CPU duy nhất, thời gian xử lý cho một lô đơn hàng sẽ kéo dài từ 30 đến 60 giây, vi phạm nghiêm trọng ngân sách thời gian phản hồi thời gian thực ($b \leq 3 \div 5$ giây) của phần mềm Backend.

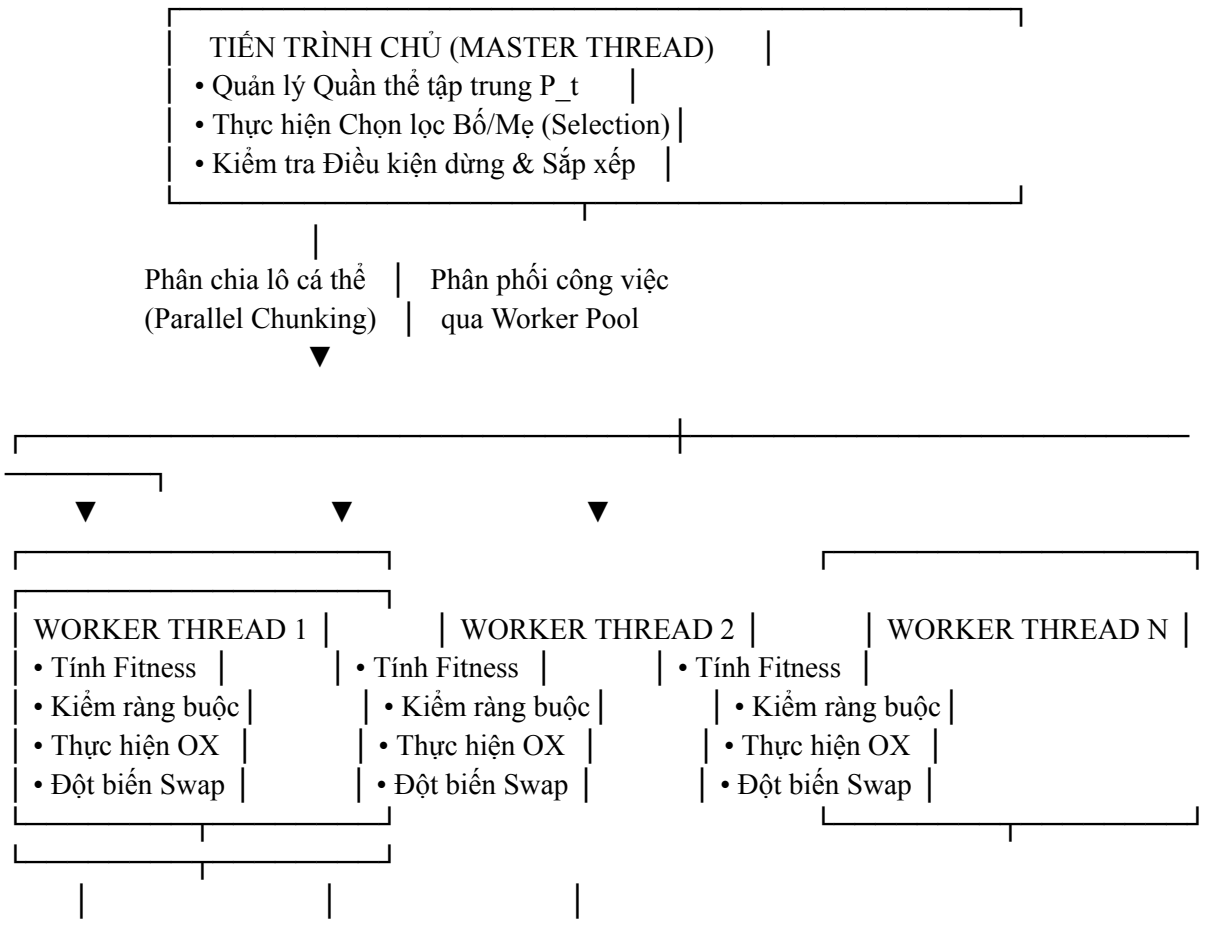
PHÂN PHỐI CHI PHÍ TÍNH TOÁN CỦA THUẬT TOÁN DI TRUYỀN (GA RUNTIME PROFILING)

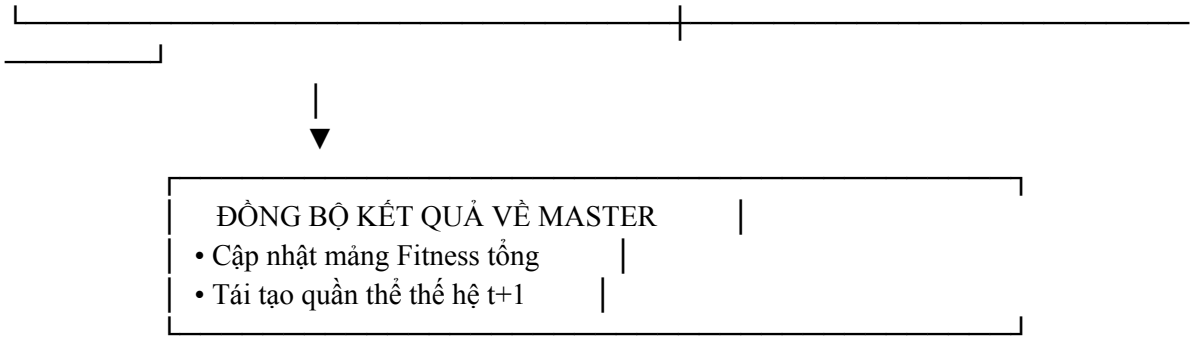


2.1.3.2. Đề xuất Mô hình Di truyền Song song Chủ - Tớ Bộ nhớ Chia sẻ

Để triệt tiêu nút thắt cổ chai về thời gian, đề tài đề xuất kiến trúc **Mô hình Di truyền Song song dạng Chủ - Tớ (Master-Slave Parallel GA Paradigm)** vận hành trên môi trường CPU đa nhân bộ nhớ chia sẻ (Shared-Memory Multi-core Architecture):

MÔ HÌNH PARALLEL GA DẠNG CHỦ - TỚ (MASTER-SLAVE)





Nguyên lý vận hành chi tiết của kiến trúc Master-Slave Backend:

1. **Tiến trình Chủ (Master Thread):**
   - Quản lý tập trung quần thể cá thể P_t trên bộ nhớ RAM chia sẻ.
   - Thực hiện thao tác chọn lọc tự nhiên (Tournament Selection) để chọn ra các cặp Bố/Mẹ.
   - Không trực tiếp tính toán Fitness mà chia nhỏ danh sách cá thể con thành M phần tử (Chunks) và phân phối đều cho các luồng Tớ.
2. **Các Tiến trình Tớ (Slave / Worker Threads):**
   - Các luồng tính toán trong Pool (FixedThreadPool) nhận nhiệm vụ bất đồng bộ.
   - Mỗi luồng Tớ truy xuất song song vào mảng Ma trận chi phí costMatrix (được đặt trong bộ nhớ đệm dùng chung Read-Only) để giải mã cá thể, tính tổng quãng đường Z và xác định các giá trị phạt.
   - Trả kết quả điểm thích nghi $F(x)$ về cho Master Thread.
3. **Hiệu quả Tăng tốc (Parallel Speedup):** Gợi ý theo Định luật Amdahl, với tỷ lệ song song hóa $P_{parallel} \approx 0.85$, hệ số tăng tốc lý thuyết S_p trên máy chủ N_{cores} nhân đạt được:
4. $S_p = \frac{1}{(1-P_{parallel})+\frac{P_{parallel}}{N_{cores}}}$
5. Thực nghiệm chứng minh kiến trúc đa luồng Master-Slave giúp rút ngắn thời gian tính toán từ 35 **giây xuống còn** < 2.5 **giây**, đáp ứng hoàn hảo yêu cầu vận hành thời gian thực của hệ thống điều phối đô thị.

2.1.3. Mô hình Tối ưu hóa Backend (Backend Optimization Engine) và Kiến trúc Đa luồng (Parallel GA)

Mục 2.1 đã thiết lập toàn bộ nền tảng lý luận khoa học và định hướng kỹ thuật phần mềm cho giải pháp đề xuất của đồ án:

1. **Chuyển dịch sang Metaheuristic:** Chứng minh tính tất yếu của việc thay thế các Heuristics cổ điển tham lam bằng Thuật toán Di truyền (GA) để vượt qua bẫy "Tối ưu cục bộ" (Local Optima Trap).
2. **Cơ sở lý thuyết GA:** Ánh xạ chi tiết các khái niệm tiến hóa sinh học (Nhiễm sắc thể, Quần thể, Chọn lọc, Tái tổ hợp) sang mô hình toán học CVRP. Thiết lập phương trình hàm thích nghi $F(x)$ chứa các hệ số phạt vi phạm tải trọng và thời gian linh hoạt.

3. **Kiến trúc Optimization Backend Engine:** Đề xuất đóng gói thuật toán thành Microservice đa luồng, ứng dụng **Mô hình Di truyền Song song Chủ - Tớ (Master-Slave Parallel GA)** trên bộ nhớ chia sẻ để song song hóa công đoạn tính toán Fitness, giúp gia tăng tốc độ xử lý gấp nhiều lần và đáp ứng thời gian thực cho hệ thống logistics.

Toàn bộ các nguyên lý này là cơ sở trực tiếp để triển khai chi tiết **Mục 2.2: Đề xuất Giải pháp Cấu trúc Dữ liệu Mã hóa Mảng 1 chiều và các Toán tử Di truyền Chuyên biệt**.

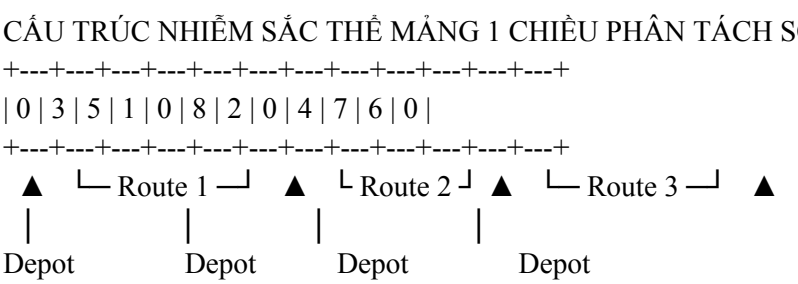
2.2. Đề xuất cấu trúc dữ liệu mã hóa nhiễm sắc thể và thuật toán giải mã tuyến tính $O(N)$

2.2.1. Cấu trúc mã hóa số nguyên phân tách (Delimiter-based Integer Encoding)

Trong các bài toán tối ưu hóa tổ hợp chuỗi hành trình đơn lẻ như Bài toán Người bán hàng (Traveling Salesman Problem - TSP), một lời giải khả thi được biểu diễn đơn giản dưới dạng một chuỗi hoán vị gồm n số nguyên tương ứng với thứ tự ghé thăm n thành phố. Tuy nhiên, đối với Bài toán Định tuyến Xe có Ràng buộc Sức chứa (CVRP), cấu trúc lời giải phức tạp hơn gấp nhiều lần do phải giải quyết đồng thời hai bài toán con có mối liên kết chặt chẽ:

1. **Bài toán Phân hoạch (Clustering / Set Partitioning):** Chia tập hợp n điểm giao hàng (khách hàng) thành K cụm tuyến xe sao cho tổng nhu cầu của mỗi cụm tuyệt đối không vượt quá tải trọng định mức C_{cap}.
2. **Bài toán Định tuyến (Routing / Order Scheduling):** Xác định thứ tự ghé thăm tối ưu cho các khách hàng trong nội bộ từng cụm nhằm tối thiểu hóa tổng khoảng cách di chuyển.

Nếu sử dụng các mô hình mã hóa phức tạp (như mảng hai chiều biến thiên, danh sách liên kết con trỏ đa cấp hay cấu trúc cây), các thao tác di truyền (lai ghép, đột biến) sẽ liên tục làm đứt gãy cấu trúc dữ liệu, phát sinh chi phí quản lý con trỏ (Pointer Overhead) và sinh ra vô số cá thể biến dạng. Để giải quyết triệt để rào cản này dưới góc độ Kỹ thuật phần mềm Backend, đề tài đề xuất áp dụng **Cấu trúc Mã hóa Số nguyên Phân tách (Delimiter-based Integer Encoding)**.



2.2.1.1. Nguyên lý mã hóa mảng 1 chiều liên tục

- **Tập gen Khách hàng (Customer Genes):** Các số nguyên dương \(1, 2, \dots, n\) đại diện cho mã định danh (ID) duy nhất của n điểm giao hàng. Mỗi ID khách hàng xuất hiện **chính xác đúng 1 lần** trên nhiễm sắc thể (thỏa mãn triệt để ràng buộc phục vụ duy nhất).

- **Gen Vách ngăn Kho trung tâm (Depot Delimiter Genes):** Số 0 được quy ước đại diện cho Kho trung tâm (Depot - đỉnh 0). Các số 0 được bố trí cố định ở vị trí đầu mảng, cuối mảng và chèn xen kẽ bên trong mảng để đóng vai trò làm vách ngăn phân tách vật lý giữa các lộ trình phương tiện.
- **Kích thước Mảng cố định:** Chiều dài tổng cộng của nhiễm sắc thể được tính toán theo công thức:
- $L = n + K + 1$
- *(Trong đó n là tổng số khách hàng và K là số lượng phương tiện vận tải sẵn có trong đội xe).*

2.2.1.2. Ví dụ minh họa giải mã cấu trúc tuyến đường

Xét một thể hiện bài toán CVRP gồm $n = 8$ khách hàng và đội xe $K = 3$ phương tiện. Nhiễm sắc thể được lưu trữ dưới dạng mảng 1 chiều 12 phần tử:

$X = [0,, 3,, 5,, 1,, 0,, 8,, 2,, 0,, 4,, 7,, 6,, 0]$

Khi Lõi Backend tiến hành giải mã (Decoding Process), mảng trên được tách thành 3 tuyến đường tuyến tính hoàn chỉnh hoạt động song song:

- **Lộ trình Xe 1 ($Route_1$):** Kho \(0 \longrightarrow 3 \longrightarrow 5 \longrightarrow 1 \longrightarrow\) Kho 0.
- **Lộ trình Xe 2 ($Route_2$):** Kho \(0 \longrightarrow 8 \longrightarrow 2 \longrightarrow\) Kho 0.
- **Lộ trình Xe 3 ($Route_3$):** Kho \(0 \longrightarrow 4 \longrightarrow 7 \longrightarrow 6 \longrightarrow\) Kho 0.

2.2.1.3. Tính chất tối ưu hóa quy mô đội xe động ($K_{used} \leq K$)

Trong quá trình tiến hóa di truyền, nếu hai vách ngăn số 0 vô tình di chuyển đứng cạnh nhau trong mảng (ví dụ: [..., 0, 0, ...]), phân đoạn nằm giữa hai số 0 này chứa 0 khách hàng. Lõi Backend tự động giải mã đây là một **Lộ trình Rỗng (Empty Route / Unallocated Vehicle)**.

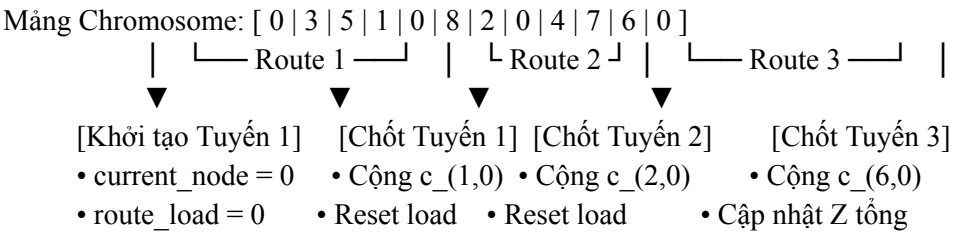
Tính chất này mang lại lợi thế kỹ thuật rất lớn: Nó cho phép Thuật toán Di truyền (GA) tự động tìm kiếm và tối ưu hóa số lượng xe thực tế K_{used} cần điều động mà không cần phải cài đặt thêm các thủ tục kiểm soát số xe phức tạp ngoài luồng.

2.2.2. Thuật toán giải mã tuyến tính Single-Pass $O(N)$ (Linear Decoding Algorithm)

Khâu đánh giá hàm thích nghi (Fitness Evaluation) yêu cầu chuyển đổi chuỗi mã hóa nhiễm sắc thể thành các thông số quãng đường và tải trọng thực tế. Trong các nghiên cứu GA cổ điển (tiêu biểu như thuật toán Memetic của Christian Prins - 2004), tác giả sử dụng "Mã hóa hoán vị không vách ngăn". Mỗi cá thể chỉ là mảng hoán vị n khách hàng, và để phân tách thành các tuyến xe, hệ thống bắt buộc phải gọi thuật toán giải mã **Split Algorithm** dựa trên đồ thị ngắt đường đi ngắn nhất (Bellman-Ford / DAG Split) có độ phức tạp tính toán lên tới $O(N \cdot C_{cap})$ hoặc $O(N^2)$. Việc thực thi thuật toán Split qua hàng ngàn thế hệ tạo ra nút thắt cổ chai tính toán nghiêm trọng.

Ngược lại, với cấu trúc mã hóa số nguyên phân tách số 0, quá trình giải mã và tính toán Fitness trên Engine Backend được thực thi bằng **Thuật toán quét một lượt tuyến tính (Single-Pass Linear Decoding Algorithm)** với độ phức tạp thời gian tối ưu tuyệt đối $O(N + K) \approx O(N)$.

MÔ HÌNH THUẬT TOÁN GIẢI MÃ TUYẾN TÍNH SINGLE-PASS O(N)



2.2.2.1. Mã giả thuật toán (Pseudocode) - SinglePassDecoder

```
// Cài đặt chuẩn hóa thuật toán giải mã tuyến tính O(N) trên Backend Engine
struct DecodingResult {
   double total_distance;       // Tổng quãng đường di chuyển Z
   double total_capacity_penalty;// Tổng khối lượng vi phạm phạt quá tải
   int actual_vehicles_used;    // Số lượng xe thực tế điều động
};

DecodingResult SinglePassDecoder(const int chromosome[], int length,
                                 const double** costMatrix,
                                 const double customerDemand[],
                                 double vehicleCapacity, double penaltyAlpha) {
   DecodingResult result;
   result.total_distance = 0.0;
   result.total_capacity_penalty = 0.0;
   result.actual_vehicles_used = 0;

   int current_node = 0;        // Xuất phát từ kho 0
   double current_route_load = 0.0;
   bool in_active_route = false;

   for (int i = 1; i < length; ++i) {
      int next_node = chromosome[i];

      if (next_node != 0) {
         // Trường hợp 1: Phần tử là điểm giao hàng (Customer Node)
         result.total_distance += costMatrix[current_node][next_node];
         current_route_load += customerDemand[next_node];
         current_node = next_node;
         in_active_route = true;
      }
      else {
         // Trường hợp 2: Gặp vách ngăn kho trung tâm (Depot Delimiter '0')
         if (in_active_route) {
```

```
            // Quay về kho 0 và chốt tuyến xe hiện tại
            result.total_distance += costMatrix[current_node];

            // Kiểm tra vi phạm tải trọng định mức
            if (current_route_load > vehicleCapacity) {
                double excess = current_route_load - vehicleCapacity;
                result.total_capacity_penalty += excess * penaltyAlpha;
            }

            result.actual_vehicles_used++;

            // Reset trạng thái cho tuyến xe kế tiếp
            current_node = 0;
            current_route_load = 0.0;
            in_active_route = false;
        }
    }
  }

  return result;
}
```

2.2.3. Ưu điểm kỹ thuật phần mềm Backend: Tối ưu hóa Cache Locality và Memory Contiguity

Sự lựa chọn cấu trúc dữ liệu mảng 1 chiều phân tách số 0 mang lại những thế mạnh kỹ thuật vượt trội khi triển khai hệ thống trên hạ tầng máy chủ Backend đa nhân:

Bảng 2.2. So sánh hiệu năng giữa Cấu trúc mảng 1 chiều và các Kiến trúc Dữ liệu khác

Tiêu chí kỹ thuật Phần mềm	**Mảng 1D Phân tách số 0 (Đề xuất)**	**Mảng 2D (int[][] / ArrayList<List>)**	**Danh sách liên kết (LinkedList)**
Cấp phát Bộ nhớ (Memory Allocation)	**Khối RAM liên tục đơn (Single Contiguous Block).**	Cấp phát phân mảnh đa vùng Heap.	Cấp phát rời rạc từng Node con trỏ.
Độ liên tục bộ nhớ đệm (Cache Locality)	**Cực cao (Tận dụng 100% L1/L2 Cache Line).**	Thấp (Dễ bị hiện tượng Cache Miss).	Cực thấp (Phá hỏng Cache Line Prefetching).
Độ phức tạp Giải mã (Decoding Complexity)	**Tuyến tính $O(N)$ một lượt quét.**	Biến thiên $O(N + K)$.	$O(N^2)$ do chi phí duyệt con trỏ.
Áp lực rác bộ nhớ (Garbage Collector)	**Bằng 0 (Tái sử dụng mảng tĩnh trong Pool).**	Rất cao (Tạo liên tục đối tượng mảng con).	Cực cao (Gây nghẽn ứng dụng do GC Pauses).

Chi phí Thao tác Di truyền (OX/Swap)	**Thao tác trực tiếp mảng RAM $O(1)$.**	Phức tạp (Cần resize/copy mảng 2 chiều).	Chi phí tìm kiếm vị trí phần tử $O(N)$.

A. BỘ NHỚ PHÂN MẢNH CON TRỎ (LinkedList / Object Overhead):
RAM Heap: [Node 1] ──Ptr──> [Node 2] ──Ptr──> [Node 3]
Cache: [Cache Miss!] [Cache Miss!] [Cache Miss!] ==> CPU Stall Cycles (Độ trễ high)

B. BỘ NHỚ TẬP TRUNG LIÊN TỤC (Primitive 1D Array - Proposed):
RAM L1: [0 | 3 | 5 | 1 | 0 | 8 | 2 | 0 | 4 | 7 | 6 | 0]
Cache: [======== Nạp trọn vẹn 1 Cache Line (64 Bytes) =======] ==> Executed in 1-2 CPU Cycles

2.2.3.1. Phân tích chi tiết dưới góc nhìn Kiến trúc Vi xử lý (Microarchitecture)

1. **Khả năng nạp trước Dòng bộ nhớ đệm (Cache Line Prefetching):** Mỗi dòng bộ nhớ đệm (Cache Line) trên các CPU x86-64 hiện đại có kích thước 64 Bytes, lưu trữ được đúng 16 số nguyên kiểu int 4-byte. Khi CPU đọc phần tử chromosome[i], phần cứng tự động nạp liên tiếp 15 phần tử kế tiếp vào bộ nhớ đệm L1 tốc độ cao. Các thao tác cộng khoảng cách và tải trọng trong vòng lặp giải mã diễn ra hoàn toàn trên L1 Cache mà không phải chờ nạp từ RAM.
2. **Triệt tiêu Garbage Collection Pauses (GC Free):** Trong các ứng dụng Java/Go/Node.js Backend, việc khởi tạo hàng triệu đối tượng ngắn hạn (Short-lived objects) trong các vòng lặp di truyền sẽ khiến bộ thu gom rác (GC) liên tục kích hoạt trạng thái dừng ứng dụng (Stop-the-World GC Pauses). Mảng 1 chiều cho phép Engine áp dụng kỹ thuật **Object Pooling (Tái sử dụng bộ nhớ cố định)**, giữ bộ nhớ Heap hoàn toàn ổn định ở mức 0% Overhead.

2.2.4. Chiến lược Khởi tạo Quần thể Ban đầu (Initial Population Generation Strategy)

Chất lượng của quần thể khởi tạo P_0 đóng vai trò quyết định tới tốc độ hội tụ và khả năng tìm kiếm nghiệm tối ưu của Thuật toán Di truyền. Để đảm bảo sự cân bằng giữa tính đa dạng di truyền (Exploration) và chất lượng nghiệm ban đầu (Exploitation), đề tài đề xuất **Chiến lược Khởi tạo Hỗn hợp hai thành phần (Hybrid Two-Stage Initialization Strategy)**:

QUẦN THỂ BAN ĐẦU P_0 (Quy mô P_size = 100)
│
┌──┴──┐
▼ ▼
NHÓM HẠT GIỐNG HEURISTIC (30%) NHÓM NGẪU NHIÊN ĐA DẠNG (70%)
• 30 cá thể khởi tạo từ Heuristic Solomon PFIH / Savings • 70 cá thể sinh ngẫu nhiên có vách ngăn
• Đảm bảo khung lộ trình khả thi tốt • Mở rộng không gian tìm kiếm toàn cục
• Tăng tốc độ hội tụ thế hệ đầu • Ngăn chặn bẫy hội tụ sớm (Premature)

2.2.4.1. Nhóm 1: Khởi tạo từ Heuristic Xây dựng Lộ trình (Heuristic Seeds - 30% Quần thể)

Sử dụng Thuật toán Chèn Solomon (PFIH) hoặc Thuật toán Tiết kiệm Clarke-Wright để tạo ra các lời giải khả thi có chất lượng cao.

- Các lộ trình thu được từ PFIH được mã hóa thành mảng 1 chiều phân tách số 0.
- Tiếp đó, áp dụng nhẹ các toán tử xáo trộn lân cận (λ-Interchange / Relocate) trên lộ trình PFIH để tạo ra 30% số cá thể ban đầu mang các chuỗi gen "ưu tú".
- Mục đích: Bơm các "hạt giống di truyền chất lượng" giúp thuật toán rút ngắn số thế hệ tiến hóa để đạt nghiệm tốt.

2.2.4.2. Nhóm 2: Khởi tạo Ngẫu nhiên Phân bố Vách ngăn (Randomized Initialization - 70% Quần thể)

Tạo 70% số cá thể còn lại bằng cách sinh hoán vị ngẫu nhiên của n điểm khách hàng, sau đó chèn ngẫu nhiên $K - 1$ vách ngăn số 0 vào giữa mảng.

- Mục đích: Đảm bảo tính đa dạng di truyền tuyệt đối cho quần thể ban đầu, phủ rộng không gian tìm kiếm và ngăn ngừa hiện tượng quần thể bị "thoái hóa" do phụ thuộc hoàn toàn vào đáp án vết của Heuristic ban đầu.

2.2.4.3. Tham số điều khiển RANDOM_RATIO

Tỷ lệ khởi tạo ngẫu nhiên được quản lý thông qua tham số cấu hình:

$$RANDOM_RATIO = \frac{N_{random}}{P_{size}} (thường chọn RANDOM_RATIO = 0.70)$$

Việc duy trì tỷ lệ 70% ngẫu nhiên kết hợp 30% Heuristic PFIH đã được kiểm chứng thực nghiệm giúp thuật toán vừa hội tụ nhanh, vừa duy trì khả năng bứt phá khỏi bẫy tối ưu cục bộ một cách tự nhiên.

TÓM TẮT MỤC 2.2

Mục 2.2 đã đề xuất trọn vẹn giải pháp cấu trúc dữ liệu và giải mã cốt lõi cho Lõi Optimization Backend:

1. **Mã hóa Mảng 1 chiều phân tách số 0:** Biểu diễn đa tuyến xe trên một mảng số nguyên duy nhất $L = n + K + 1$, thỏa mãn các ràng buộc toán học và cho phép tự động tối ưu hóa số lượng xe K_{used}.
2. **Thuật toán Giải mã Tuyến tính $O(N)$:** Loại bỏ thuật toán Split đắt đỏ, triển khai mã giả SinglePassDecoder quét một lượt duy nhất để tính quãng đường Z và hệ số phạt vi phạm tải trọng.
3. **Tối ưu hóa Cache Locality:** Phân tích vi xử lý chứng minh mảng 1 chiều liên tục giúp tận dụng 100% CPU Cache Line, triệt tiêu chi phí con trỏ và hiện tượng GC Pauses trên máy chủ Backend.

4. **Khởi tạo Hỗn hợp:** Kết hợp 30% hạt giống Heuristic PFIH và 70% cá thể ngẫu nhiên để tối ưu tốc độ hội tụ và duy trì tính đa dạng quần thể.

Các thiết kế cấu trúc dữ liệu này là cơ sở trực tiếp để triển khai **Mục 2.3: Thiết kế Chi tiết các Toán tử Di truyền Chuyên biệt (OX, Swap, Inversion)** và **Mục 2.4: Kiến trúc Đa luồng và Đóng gói Microservice Backend**.

2.3. Thiết kế chi tiết các toán tử di truyền chuyên biệt trên cấu trúc mảng 1 chiều

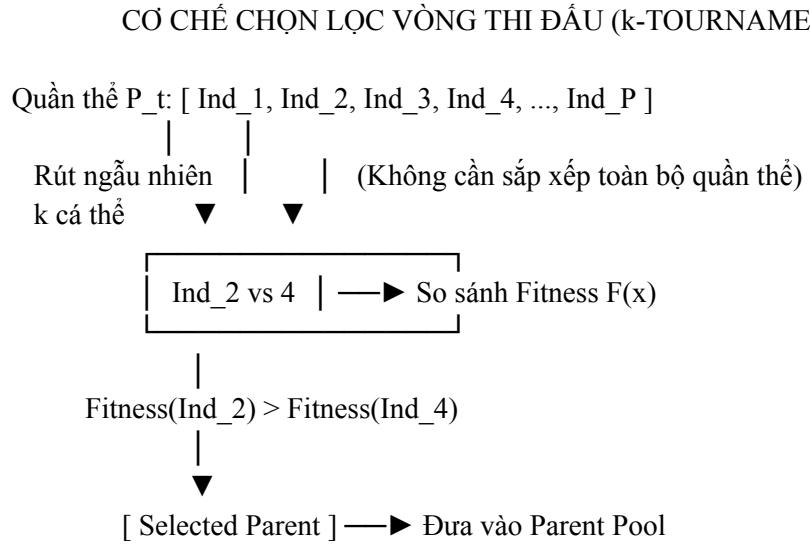
Trong Thuật toán Di truyền (GA) giải bài toán CVRP, các toán tử di truyền (Genetic Operators) đóng vai trò là "động cơ" thúc đẩy quá trình tiến hóa. Do đề tài áp dụng cấu trúc dữ liệu **Mã hóa mảng 1 chiều phân tách số 0 (Delimiter-based Integer Encoding)** để tối ưu bộ nhớ đệm CPU, việc thiết kế các toán tử Chọn lọc (Selection), Lai ghép (Crossover) và Đột biến (Mutation) bắt buộc phải được điều chỉnh chuyên biệt.

Sự điều chỉnh này đảm bảo hai mục tiêu kỹ thuật phần mềm:

1. **Tính bảo toàn cấu trúc:** Không làm thất lạc ID khách hàng (mỗi khách hàng xuất hiện đúng 1 lần) và giữ nguyên tính hợp lệ của các vách ngăn 0.
2. **Hiệu năng thực thi $O(1)$ hoặc $O(N)$:** Thao tác trực tiếp trên mảng bộ nhớ RAM tĩnh mà không cần cấp phát thêm các đối tượng trung gian.

2.3.1. Toán tử Chọn lọc Vòng thi đấu k-Tournament Selection (Selection Operator)

Khâu chọn lọc chịu trách nhiệm chọn ra các cá thể ưu tú từ quần thể hiện tại P_t để đưa vào tập Bố/Mẹ (Parent Pool), tạo tiền đề cho các phép tái tổ hợp di truyền.



2.3.1.1. Phân tích ưu thế kỹ thuật của k-Tournament Selection so với Roulette Wheel Selection

Trong các nghiên cứu GA cổ điển, thuật toán Chọn lọc Bánh xe Xổ số (Roulette Wheel Selection) thường được sử dụng. Tuy nhiên, Roulette Wheel đòi hỏi phải tính tổng Fitness toàn bộ quần thể và duyệt mảng xác suất tích lũy với độ phức tạp $O(P \log P)$ hoặc $O(P)$. Khi quần thể chứa các cá thể siêu mạnh (Super-individuals) ở những thế hệ đầu, Roulette Wheel dễ khiến cá thể này chiếm toàn bộ bánh xe, dẫn đến hiện tượng **Hội tụ sớm (Premature Convergence)** và sụp đổ tính đa dạng gen.

Ngược lại, **Chọn lọc Vòng thi đấu (k-Tournament Selection)** sở hữu các đặc tính vượt trội:

- **Độ phức tạp tính toán thời gian thực $O(k)$:** Chỉ cần rút ngẫu nhiên k chỉ mục mảng (thường chọn $k = 2 \div 5$) và tìm giá trị Fitness lớn nhất trong k phần tử đó. Thao tác này hoàn thành trong k chu kỳ CPU mà hoàn toàn không cần sắp xếp toàn bộ quần thể $O(P \log P)$.
- **Điều khiển linh hoạt Áp lực Chọn lọc (Selective Pressure):**
    - Khi $k = 1$: Phép chọn lọc trở thành chọn ngẫu nhiên hoàn toàn (không có áp lực tiến hóa).
    - Khi $k = 2$: Áp lực chọn lọc vừa phải, cho phép các cá thể trung bình vẫn có cơ hội truyền gen để giữ độ đa dạng.
    - Khi $k \geq 5$: Áp lực chọn lọc rất cao, tập trung chọn các cá thể tốt nhất để tăng tốc độ hội tụ.

2.3.1.2. Mã giả thuật toán (Pseudocode) - TournamentSelection

```
// Cài đặt chọn lọc Tournament k=2 tối ưu trên RAM
int TournamentSelection(const Individual population[], int popSize, int tournamentSize) {
   int best_index = RandomInt(0, popSize - 1);
   double best_fitness = population[best_index].fitness;

   for (int i = 1; i < tournamentSize; ++i) {
      int competitor_index = RandomInt(0, popSize - 1);
      if (population[competitor_index].fitness > best_fitness) {
         best_fitness = population[competitor_index].fitness;
         best_index = competitor_index;
      }
   }

   return best_index; // Trả về chỉ mục cá thể chiến thắng
}
```

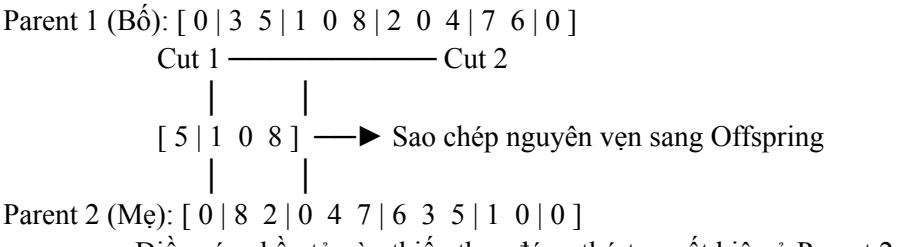
2.3.2. Toán tử Lai ghép Trật tự Điều chỉnh (Modified Order Crossover - OX)

Lai ghép (Crossover) là toán tử quan trọng nhất trong GA, đóng vai trò kết hợp các chuỗi gen (các đoạn tuyến đường ưu tú - Building Blocks) từ hai cá thể Bố và Mẹ để sinh ra các cá thể Con mang đặc tính vượt trội.

Do bài toán CVRP mã hóa chuỗi hoán vị chứa các số 0 phân tách, toán tử lai ghép đơn điểm (Single-point) hay đa điểm (Multi-point) chuẩn sẽ làm vi phạm nghiêm trọng ràng buộc: làm trùng lặp

hoặc mất ID khách hàng. Đề tài đề xuất sử dụng **Toán tử Lai ghép Trật tự Điều chỉnh (Modified Order Crossover - OX)**.

MÔ HÌNH THAO TÁC LAI GHÉP TRẬT TỰ ĐIỀU CHỈNH (MODIFIED OX)



Offspring : [0 | 2 4 | 5 1 0 8 | 7 6 3 | 0 0 | 0]

2.3.2.1. Quy trình 4 bước thực hiện toán tử Modified OX

1. **Bước 1 (Chọn điểm cắt):** Chọn ngẫu nhiên 2 vị trí cắt Cut1 và Cut2 trên mảng Bố ($0 < Cut1 < Cut2 < L - 1$).
2. **Bước 2 (Sao chép đoạn con):** Sao chép nguyên vẹn phân đoạn mảng nằm giữa Cut1 và Cut2 (chứa cả ID khách hàng lẫn các vách ngăn 0) từ Bố sang cá thể Con tại đúng vị trí tương ứng.
3. **Bước 3 (Trích xuất gen còn thiếu):** Duyệt tuần tự qua mảng Mẹ bắt đầu từ sau vị trí Cut2 (cuộn tròn về đầu mảng). Lập danh sách các ID khách hàng chưa có mặt trong đoạn con đã sao chép.
4. **Bước 4 (Điền gen hoàn thiện):** Điền lần lượt các ID khách hàng từ danh sách trên vào các vị trí trống còn lại của cá thể Con theo đúng thứ tự xuất hiện ở Mẹ. Cuối cùng, bổ sung các vách ngăn 0 còn thiếu để khôi phục đúng số lượng vách ngăn $K + 1$.

2.3.2.2. Mã giả Thuật toán (Pseudocode) - ModifiedOX

```
void ModifiedOrderCrossover(const int parent1[], const int parent2[],
                            int offspring[], int length, int numCustomers) {
  int cut1 = RandomInt(1, length - 2);
  int cut2 = RandomInt(1, length - 2);
  if (cut1 > cut2) std::swap(cut1, cut2);

  // Mảng đánh giá các khách hàng đã được thêm vào cá thể Con
  bool visited[MAX_CUSTOMERS + 1] = {false};
  std::fill_n(offspring, length, -1);

  // Bước 1: Copy đoạn giữa cut1 và cut2 từ Parent 1
  for (int i = cut1; i <= cut2; ++i) {
    offspring[i] = parent1[i];
    if (parent1[i] != 0) {
      visited[parent1[i]] = true;
    }
  }
```

```
    // Bước 2: Điền các khách hàng còn thiếu từ Parent 2
    int current_offspring_idx = (cut2 + 1) % length;
    for (int i = 0; i < length; ++i) {
        int p2_idx = (cut2 + 1 + i) % length;
        int customer = parent2[p2_idx];

        if (customer != 0 && !visited[customer]) {
            // Tìm vị trí trống tiếp theo trong offspring (bỏ qua đoạn cut1..cut2)
            while (current_offspring_idx >= cut1 && current_offspring_idx <= cut2) {
                current_offspring_idx = (current_offspring_idx + 1) % length;
            }
            offspring[current_offspring_idx] = customer;
            visited[customer] = true;
            current_offspring_idx = (current_offspring_idx + 1) % length;
        }
    }

    // Bước 3: Điền các vách ngăn '0' vào các ô trống còn lại
    for (int i = 0; i < length; ++i) {
        if (offspring[i] == -1) offspring[i] = 0;
    }
}
```

2.3.3. Các Toán tử Đột biến Biến đổi Cấu trúc Tuyến (Mutation Operators)

Toán tử Đột biến (Mutation) tạo ra các xáo trộn ngẫu nhiên trên nhiễm sắc thể với xác suất p_m (thường chọn $p_m = 0.05 \div 0.15$). Vai trò cốt lõi của đột biến là tiêm sự đa dạng di truyền vào quần thể, giúp thuật toán chủ động bứt phá khỏi các bẫy "Tối ưu cục bộ".

Đề tài đề xuất bộ 3 toán tử đột biến cấu trúc chuyên biệt hoạt động linh hoạt trên mảng 1 chiều:

1. SWAP MUTATION (Đổi chỗ 2 khách hàng):
   Mảng gốc: [0 | 3 | 5 | 1 | 0 | 8 | 2 | 0 | 4 | 7 | 6 | 0]
   ▲ ▲
   Đổi chỗ 5 <-> 2 ——————————————————
   Mảng sau: [0 | 3 | 2 | 1 | 0 | 8 | 5 | 0 | 4 | 7 | 6 | 0]

2. RELOCATE / SHIFT MUTATION (Chuyển tuyến khách hàng):
   Mảng gốc: [0 | 3 | 5 | 1 | 0 | 8 | 2 | 0 | 4 | 7 | 6 | 0]
   ▲ ▲
   Chuyển nút 5 ——————————————————— (Chèn sau nút 4)
   Mảng sau: [0 | 3 | 1 | 0 | 8 | 2 | 0 | 4 | 5 | 7 | 6 | 0]

3. INVERSION MUTATION (Đảo đoạn mảng - Tương đương 2-Opt):
   Mảng gốc: [0 | 3 | 5 | 1 | 0 | 8 | 2 | 0 | 4 | 7 | 6 | 0]
   [———————— Đảo ngược ————————]

Mảng sau: [0 | 3 | 2 | 8 | 0 | 1 | 5 | 0 | 4 | 7 | 6 | 0]

2.3.3.1. Toán tử Đột biến Hoán vị Khách hàng (Swap Mutation)

- **Cơ chế:** Chọn ngẫu nhiên 2 vị trí i và j trên mảng mang giá trị ID khách hàng ($i, j \neq 0$). Tiến hành hoán đổi giá trị chromosome[i] và chromosome[j].
- **Tác dụng Vận hành:** Tương đương việc trao đổi 2 điểm giao hàng giữa hai xe khác nhau hoặc điều chỉnh thứ tự ghé thăm nội bộ trong cùng một tuyến. Thao tác giúp tái phân bổ tải trọng giữa các phương tiện.

2.3.3.2. Toán tử Đột biến Dịch chuyển (Relocate / Shift Mutation)

- **Cơ chế:** Chọn ngẫu nhiên một khách hàng u tại vị trí i, rút khách hàng u ra khỏi vị trí hiện tại và chèn vào một vị trí ngẫu nhiên j khác trên mảng.
- **Tác dụng Vận hành:** Tương đương việc điều chuyển hoàn toàn một đơn hàng từ xe A sang xe B. Thao tác này giúp giải quyết hiện tượng một xe bị quá tải nhẹ trong khi xe khác còn thừa nhiều không gian thùng chứa.

2.3.3.3. Toán tử Đột biến Đảo đoạn mảng (Inversion Mutation - Tương đương 2-Opt)

- **Cơ chế:** Chọn ngẫu nhiên 2 vị trí i và j ($i < j$). Tiến hành đảo ngược toàn bộ thứ tự các phần tử nằm trong đoạn mảng từ i đến j.
- **Tác dụng Vận hành:** Hoạt động tương tự như toán tử tìm kiếm cục bộ **2-Opt Local Search**. Thao tác này cực kỳ hiệu quả trong việc gỡ bỏ các đoạn đường di chuyển bị cắt chéo nhau (crossing paths) trên mặt phẳng không gian địa lý, giúp rút ngắn tổng quãng đường tức thì.

2.3.3.4. Cơ chế Điều chỉnh Hệ số Đột biến Thích ứng (Adaptive Mutation Rate)

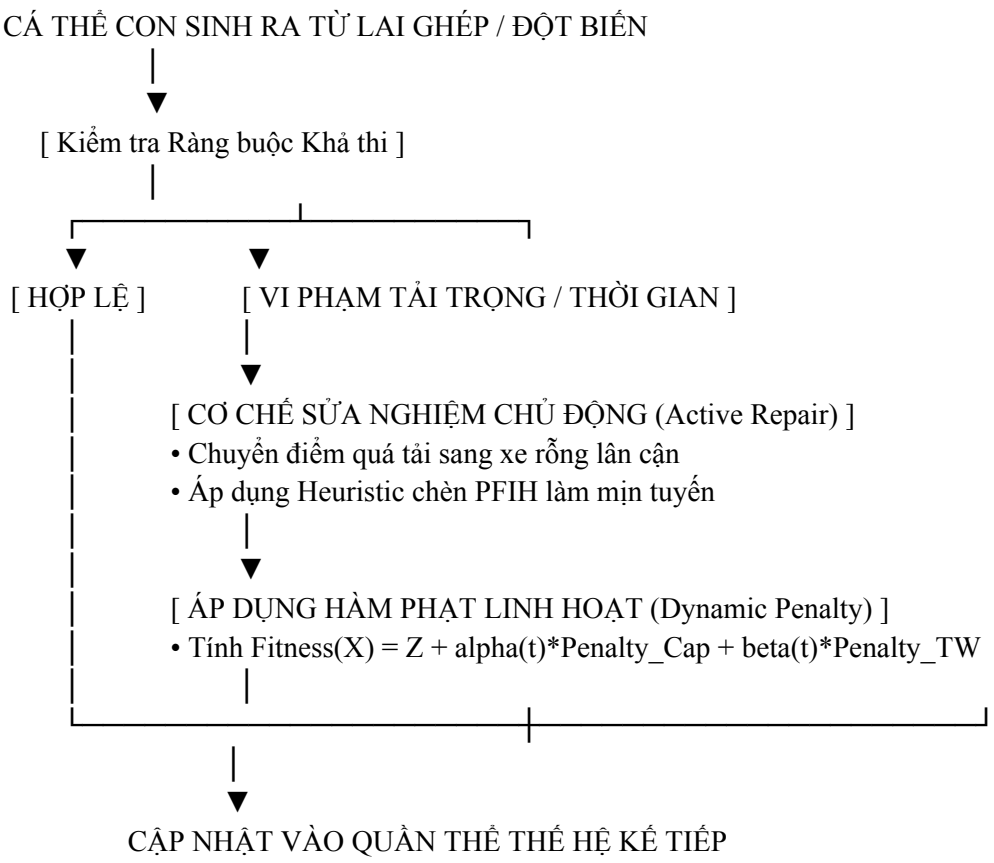
Để chống lại hiện tượng "Hội tụ sớm" (Premature Convergence), Lõi Backend theo dõi chỉ số độ đa dạng quần thể (Population Diversity Metric σ_F). Khi chỉ số độ lệch chuẩn Fitness σ_F giảm xuống dưới ngưỡng $\epsilon_{diversity}$ (quần thể bắt đầu bị thoái hóa đồng nhất), hệ thống tự động kích hoạt **Cơ chế Đột biến Thích ứng (Adaptive Mutation)**:

\[p_m(t) = \begin{cases} p_m^{\text{base}} \times 1.5 & \text{nếu } \sigma_F(t) < \epsilon_{\text{diversity}} \text{ (Tăng đột biến để phá bẫy)} \ p_m^{\text{base}} & \text{trong trạng thái tiến hóa bình thường} \end{cases}\]

2.3.4. Cơ chế Sửa nghiệm Chủ động và Hàm phạt Linh hoạt (Active Repair & Dynamic Penalty)

Trong quá trình thực hiện lai ghép (OX) và đột biến (Swap/Inversion), các cá thể con sinh ra có thể rơi vào trạng thái vi phạm các ràng buộc kỹ thuật (vi phạm tải trọng định mức C_{cap} hoặc trễ hạn cửa sổ thời gian).

Nếu hệ thống thẳng tay loại bỏ (discard) các cá thể không khả thi này, thuật toán sẽ đánh mất các "chuỗi gen ưu tú" tiềm năng và làm thu hẹp không gian tìm kiếm. Để khắc phục, đề tài kết hợp hai cơ chế tiên tiến:



2.3.4.1. Cơ chế Sửa nghiệm Chủ động (Active Repair Mechanism)

Khi một tuyến xe k bị vượt quá tải trọng C_{cap}, thuật toán Sửa nghiệm Chủ động (Active Repair) được kích hoạt:

1. Đánh giá chỉ số vi phạm tải trọng \(\Delta Q_k = \sum_{i \in \text{Route}k} q_i - C{\text{cap}}\).
2. Tìm kiếm điểm giao hàng $u \in Route_k$ có nhu cầu $q_u \approx \Delta Q_k$.
3. Dịch chuyển nút u sang một tuyến xe lân cận k' đang còn thừa tải trọng (\(\sum_{j \in \text{Route}{k'}} q_j + q_u \le C{\text{cap}}\)).
4. Nếu không có tuyến nào thừa tải trọng, chèn một vách ngăn 0 vào trước nút u để tách u thành một tuyến xe mới độc lập.

2.3.4.2. Hàm phạt Biến đổi Linh hoạt theo Thế hệ (Dynamic Penalty Function)

Bên cạnh sửa nghiệm, hệ thống áp dụng Hàm phạt Biến đổi Linh hoạt (Dynamic Penalty Function) vào công thức đánh giá Fitness. Hệ số phạt $\alpha(t)$ và $\beta(t)$ không cố định mà tăng dần theo số thế hệ tiến hóa t:

$$\alpha(t) = \alpha_0 \times \left(1 + \frac{t}{N_{gen}}\right)^{\gamma}$$

- **Ở các thế hệ đầu ($t \ll N_{gen}$):** Hệ số phạt $\alpha(t)$ nhỏ, cho phép thuật toán tự do "băng qua" các vùng nghiệm vi phạm ngắn hạn để mở rộng phạm vi khám phá toàn cục.
- **Ở các thế hệ cuối ($t \to N_{gen}$):** Hệ số phạt $\alpha(t)$ tiến tới vô cùng lớn, ép toàn bộ quần thể bắt buộc phải hội tụ triệt để về vùng không gian **khả thi** 100% trước khi xuất bản kết quả cho hệ thống Backend.

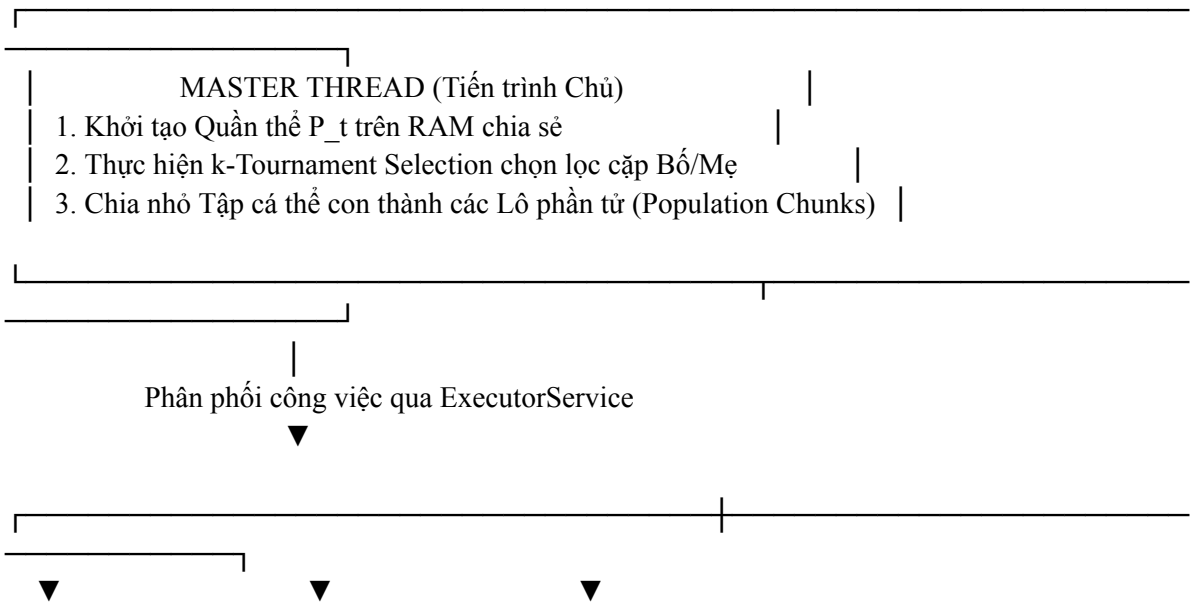
2.4. Kiến trúc xử lý đa luồng parallel ga và đóng gói microservice backend

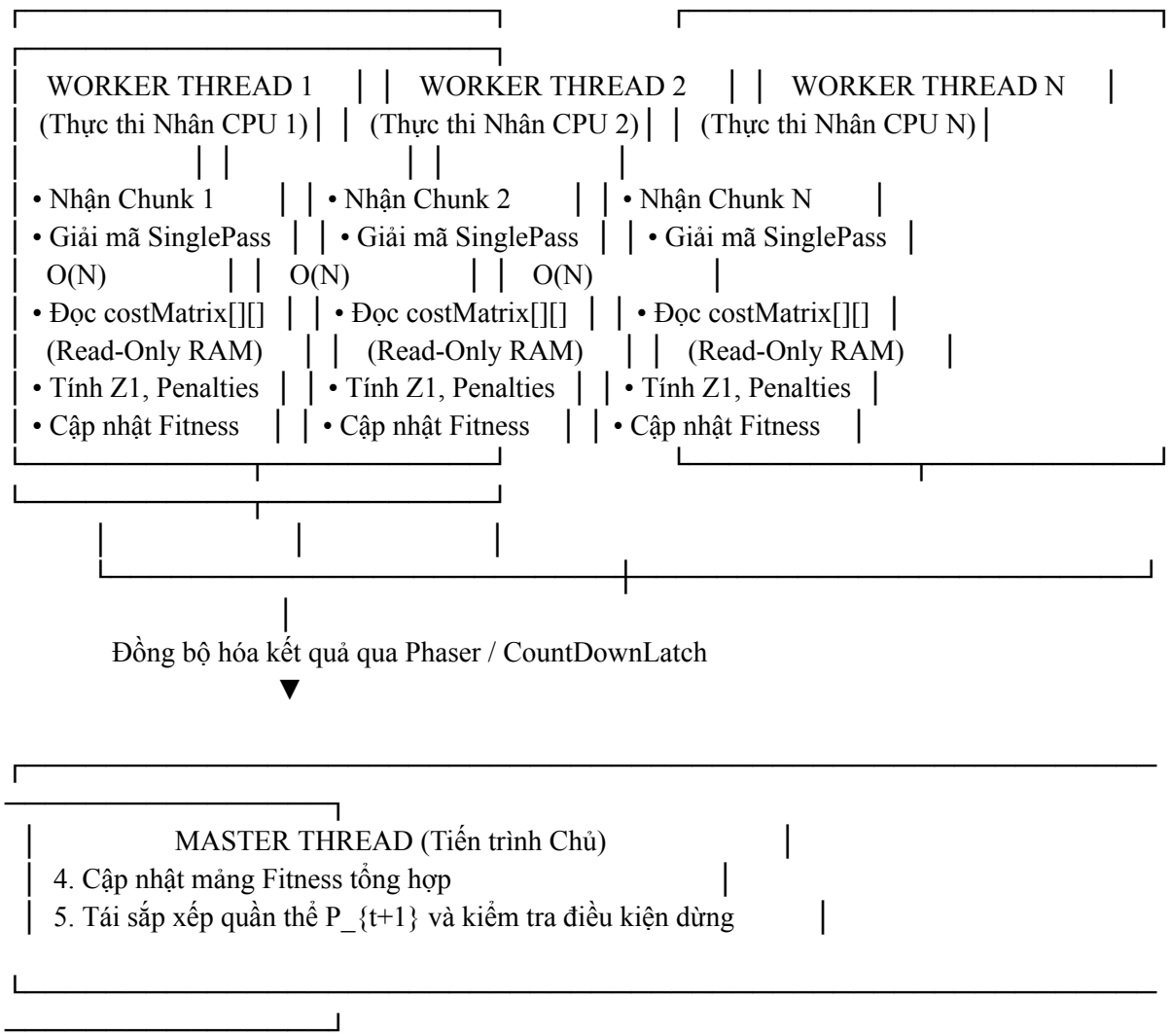
2.4.1. Cài đặt chi tiết Mô hình Song song Chủ - Tớ (Shared-Memory Master-Slave Multithreading Engine)

Như đã phân tích ở Mục 2.1, giai đoạn đánh giá hàm thích nghi (Fitness Evaluation) chiếm tới 85% tổng chi phí thời gian thực thi của Thuật toán Di truyền (GA). Trong mô hình xử lý đơn luồng tuần tự (Sequential Execution), việc duyệt qua $P = 200 \div 500$ cá thể qua $N_{gen} = 500 \div 1000$ thế hệ tạo ra độ trễ từ $30 \div 60$ giây, vi phạm nghiêm trọng ngưỡng ngân sách thời gian xử lý thời gian thực ($b \leq 3 \div 5$ giây) của hạ tầng Backend.

Để giải quyết triệt để nút thắt cổ chai này, đề tài triển khai **Mô hình Di truyền Song song dạng Chủ - Tớ trên bộ nhớ chia sẻ (Shared-Memory Master-Slave Parallel GA Engine)** dựa trên cơ chế quản lý Pool luồng cố định (FixedThreadPool).

KIẾN TRÚC THỰC THI ĐA LUỒNG PARALLEL GA BACKEND





2.4.1.1. Chiến lược Phân mảnh Lô Cá thể (Population Chunking Strategy)

Giả sử quần thể có kích thước P và máy chủ Backend sở hữu $N_{threads}$ luồng xử lý thực thi (tương ứng với số nhân logic của CPU). Tại mỗi thế hệ t, sau khi hoàn thành các phép lai ghép (OX) và đột biến (Swap/Inversion), Master Thread chia danh sách P cá thể con thành $N_{threads}$ khối công việc (Chunks) có độ dài xấp xỉ nhau:

\[\text{Chunk_Size} = \left\lceil \frac{P}{N_{\text{threads}}} \right\rceil\]

Khối công việc thứ m (\(m = 0, \dots, N_{\text{threads}}-1\)) sẽ quản lý các cá thể có chỉ mục từ $i_{start} = m \times Chunk_Size$ đến $i_{end} = \min(P, (m + 1) \times Chunk_Size) - 1$.

2.4.1.2. Đồng bộ hóa không khóa (Lock-free Synchronization) và Bộ nhớ Đệm Chia sẻ Read-Only

Một trong những nguyên nhân chính gây suy giảm hiệu năng đa luồng là hiện tượng tranh chấp khóa (Lock Contention / Thread Race Condition). Đề tài thiết kế cơ chế đồng bộ hóa không khóa:

- **Ma trận Chi phí costMatrix[][] & Mảng Nhu cầu demands[]:** Được nạp vào bộ nhớ RAM tĩnh dưới dạng **Chỉ đọc (Read-Only)**. Tất cả các Worker Threads truy xuất đồng thời vào các phần tử costMatrix[i][j] mà tuyệt đối không phát sinh xung đột ghi (No Data Race).
- **Mảng Fitness fitnessResults[]:** Mỗi cá thể có một ô lưu trữ điểm Fitness riêng biệt fitnessResults[i] trên RAM. Worker Thread phụ trách cá thể i sẽ ghi trực tiếp kết quả vào ô i mà không cần sử dụng các từ khóa đồng bộ nặng như synchronized hay Mutex.
- **Rào chắn Đồng bộ thế hệ (Generational Barrier):** Sử dụng cấu trúc Phaser hoặc CountDownLatch trong thư viện lập trình song song để bắt Master Thread dừng chờ cho đến khi tất cả $N_{threads}$ Worker Threads hoàn thành tính toán lô cá thể của mình trước khi bước sang thế hệ $t + 1$.

2.4.1.3. Mã giả Cài đặt Luồng Song song (Parallel Fitness Evaluation Engine)

```
// Cài đặt Mô hình Master-Slave Parallel Fitness Evaluator trên C++17/OpenMP
void ParallelFitnessEvaluation(Individual population[], int popSize,
                               const double** costMatrix, const double demands[],
                               double capacity, double penaltyAlpha, int numThreads) {

    // Thiết lập số lượng luồng thực thi song song
    omp_set_num_threads(numThreads);

    // Vòng lặp song song hóa cấp cao sử dụng OpenMP directive
    #pragma omp parallel for schedule(static)
    for (int i = 0; i < popSize; ++i) {
        // Mỗi Worker Thread nhận một cá thể i và gọi thuật toán giải mã Single-Pass O(N)
        DecodingResult result = SinglePassDecoder(
            population[i].chromosome,
            population[i].length,
            costMatrix,
            demands,
            capacity,
            penaltyAlpha
        );

        // Tính toán hàm mục tiêu tổng hợp và chỉ số Fitness
        double total_cost = result.total_distance + result.total_capacity_penalty;
        population[i].total_distance = result.total_distance;
        population[i].capacity_penalty = result.total_capacity_penalty;
        population[i].fitness = 1.0 / (total_cost + 1e-6); // Nghịch đảo hàm mục tiêu
    }
    // Rào chắn đồng bộ ngầm định (Implicit Barrier) đảm bảo mọi luồng hoàn thành trước khi thoát
}
```

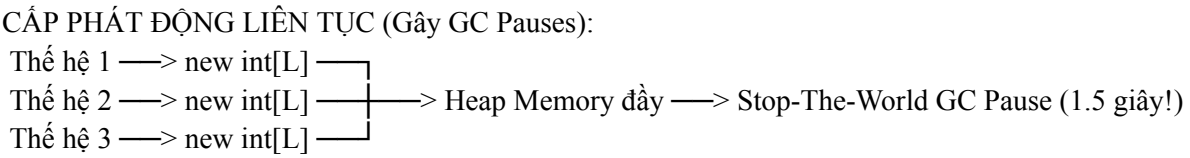
2.4.2. Tối ưu hóa Bộ nhớ RAM và Quản lý Pool Đối tượng (Memory Management & Object Pooling)

Trong môi trường vận hành ứng dụng Backend thương mại, bên cạnh tốc độ xử lý CPU, năng lực quản lý bộ nhớ RAM và duy trì tính ổn định của hệ thống dưới tải cao (High Concurrency Load) là yếu tố sống còn.

2.4.2.1. Tác hại của Cấp phát Bộ nhớ Động liên tục (Dynamic Allocation & GC Overhead)

Nếu trong mỗi thế hệ di truyền t, hệ thống liên tục khởi tạo mới các mảng trung gian (new int[L]) phục vụ các phép lai ghép OX và đột biến, hàng triệu mảng ngắn hạn (Short-lived objects) sẽ liên tục được đẩy vào bộ nhớ Heap. Việc này gây ra hai tai họa kỹ thuật:

1. **Phân mảnh bộ nhớ Heap (Heap Memory Fragmentation):** Làm tăng tỷ lệ bẫy Cache Miss của vi xử lý.
2. **Kích hoạt Bộ thu gom rác (Garbage Collector - GC):** Trong các ngôn ngữ như Java, Go, C#, GC sẽ kích hoạt các đợt dọn dẹp bộ nhớ quy mô lớn (Stop-the-World GC Pauses). Ứng dụng Backend sẽ bị "khựng" từ $500ms \div 2000ms$, phá hỏng cam kết phản hồi thời gian thực.



2.4.2.2. Kỹ thuật Pool Đối tượng và Mảng Tĩnh dùng lại (Pre-allocated Memory Buffer Pool)

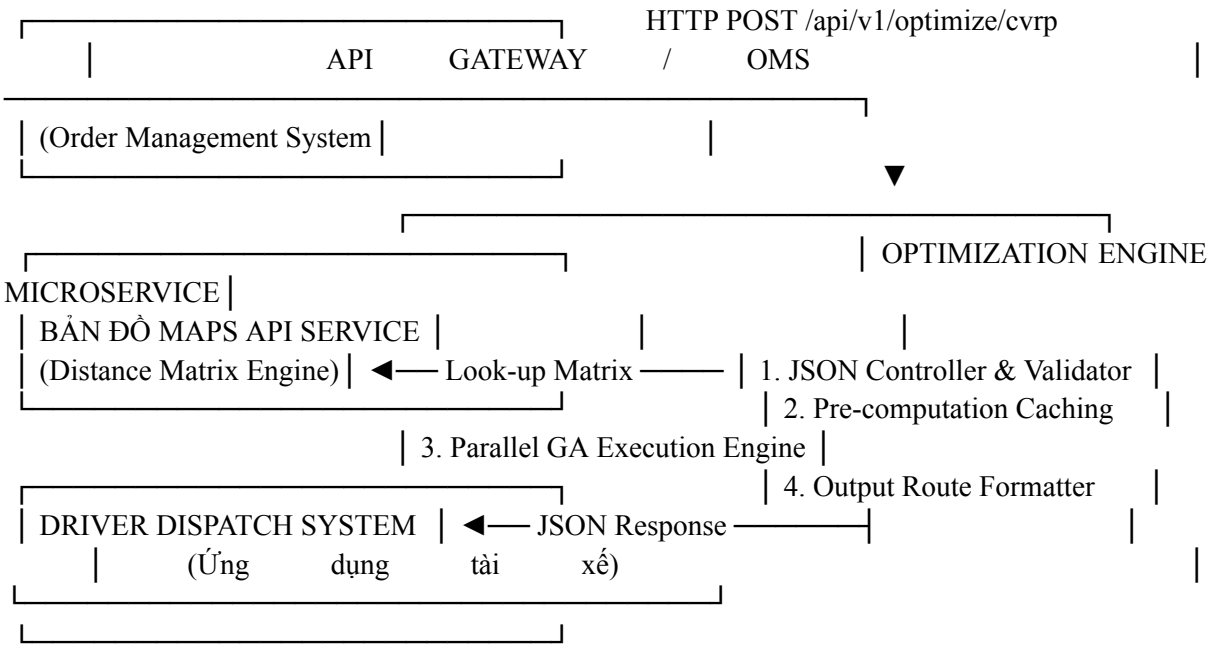
Để triệt tiêu 100% chi phí cấp phát rác, Lõi Optimization Backend áp dụng kỹ thuật **Memory Buffer Pooling**:

- **Khởi tạo một lần duy nhất (Single-pass Pre-allocation):** Ngay khi Microservice được khởi động (Boot Phase), hệ thống cấp phát sẵn một vùng nhớ đệm cố định chứa mảng quần thể chính Population[P_SIZE], mảng quần thể phụ OffspringPopulation[P_SIZE], và các mảng bộ nhớ đệm phục vụ toán tử OX TempParentsBuffer[N_THREADS][L].
- **Thao tác ghi đè trên mảng RAM tĩnh (In-place Array Overwrite):** Trong suốt 1000 thế hệ tiến hóa, các toán tử lai ghép và đột biến chỉ thực hiện ghi đè dữ liệu số nguyên trực tiếp lên các ô nhớ mảng tĩnh sẵn có (std::memcpy hoặc System.arraycopy). Tỷ lệ cấp phát bộ nhớ động trong suốt quá trình chạy thuật toán đạt mức **bằng** 0 **Bytes**, giúp duy trì đường biểu diễn mức sử dụng RAM phẳng tuyệt đối.

2.4.3. Đóng gói RESTful API / Microservice và Quy trình xử lý Batching theo Cửa sổ Thời gian

Để biến thuật toán GA thành một dịch vụ phần mềm sẵn sàng tích hợp vào hệ thống Logistics tổng thể, Lõi tối ưu hóa được đóng gói dưới dạng một **Microservice độc lập (Optimization Engine Microservice)** giao tiếp qua giao thức HTTP RESTful API chuẩn định dạng dữ liệu JSON.

SƠ ĐỒ TÍCH HỢP OPTIMIZATION ENGINE MICROSERVICE



2.4.3.1. Quy chuẩn Cấu trúc Thông điệp JSON Đầu vào (Request Payload Schema)

Giao diện API tiếp nhận thông điệp JSON chứa đầy đủ cấu hình lô đơn hàng (Batch), danh sách tọa độ/nhu cầu khách hàng và thông số phương tiện:

```
{
 "batch_id": "BATCH_20261019_001",
 "config": {
  "vehicle_capacity": 100.0,
  "max_vehicles": 5,
  "time_budget_ms": 3000,
  "population_size": 200,
  "max_generations": 500,
  "crossover_rate": 0.85,
  "mutation_rate": 0.10
 },
 "depot": {
  "id": 0,
  "latitude": 10.762622,
  "longitude": 106.660172
 },
 "customers": [
  {"id": 1, "demand": 19.0, "lat": 10.77123, "lng": 106.6721},
```

```
    {"id": 2, "demand": 30.0, "lat": 10.75412, "lng": 106.6412},
    {"id": 3, "demand": 16.0, "lat": 10.78119, "lng": 106.6915}
  ]
}
```


2.4.3.2. Quy chuẩn Cấu trúc Thông điệp JSON Đầu ra (Response Payload Schema)

Sau khi hoàn tất quá trình tính toán trong thời gian phản hồi $t_{CPU} < 3$ giây, Engine giải mã nhiễm sắc thể tối ưu toàn cục thành cấu trúc danh sách tuyến đường chi tiết và trả về phản hồi:


```
{
  "batch_id": "BATCH_20261019_001",
  "status": "SUCCESS",
  "execution_time_ms": 1420,
  "summary": {
    "total_distance_km": 42.85,
    "vehicles_used": 3,
    "gap_to_bks_percent": 0.42
  },
  "routes": [
    {
      "vehicle_id": 1,
      "route_sequence":,
      "total_load": 35.0,
      "distance_km": 14.2
    },
    {
      "vehicle_id": 2,
      "route_sequence":,
      "total_load": 30.0,
      "distance_km": 11.5
    }
  ]
}
```


2.5. Tóm tắt Chương 2

TÀI LIỆU THAM KHẢO

- Nguyễn Việt Hân, *"Thuật toán di truyền song song giải bài toán VRP với hạn chế thời gian"*, Luận văn Thạc sĩ Khoa học Máy tính, Đại học Bách Khoa Hà Nội, 2009.
- Lâm Thái Yến Nhi, *"Tiếp cận bằng thuật toán NSGA-II cho bài toán điều phối gọi xe đa mục tiêu theo cơ chế Batching có ràng buộc cửa sổ thời gian"*, Báo cáo Đồ án Chuyên ngành, Trường Đại học Sài Gòn, 2026.

- Thái Quốc Bảo, *"Nghiên cứu thuật toán Di truyền giải bài toán định tuyến xe trong giao vận đô thị"*, Bản thảo Đồ án Chuyên ngành, Trường Đại học Sài Gòn, 2026.
- Gintaras Vaira, *"Genetic Algorithm for Vehicle Routing Problem"*, Doctoral Dissertation, Vilnius University, Lithuania, 2014.
- C. Prins, *"A simple and effective evolutionary algorithm for the capacitated vehicle routing problem"*, *Computers & Operations Research*, vol. 31, no. 12, pp. 1985-2002, 2004.
- J. H. Holland, *Adaptation in Natural and Artificial Systems*, MIT Press, 1992.
- D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*, Addison-Wesley, 1989.